

```

    items: list[OcrItem] = Field(min_length=1, max_length=500)
class AgentDecisionRequest(BaseModel):
    revision: int
    decisions: list[dict] = Field(min_length=1, max_length=1)
class Derivation(BaseModel):
    id: str = Field(min_length=1, max_length=80)
    expr: str = Field(min_length=1, max_length=500)
    unit: str = Field('', max_length=16)
    label: str = Field('', max_length=80)
class DerivationsRequest(BaseModel):
    revision: int
    derivations: list[Derivation] = Field(max_length=2000)
def create_app(data_dir=None):
    load_environment()
    store = Store(data_dir or os.getenv('ZHILIAN_DATA_DIR', str(BASE / '.zhilian')))
    app = FastAPI(title='知链', version=__version__, description='跨文档结论验证与增量修复')
    app.state.store = store
    quota.configure(store.root / 'quota.json')
    @app.middleware('http')
    async def boundary(request: Request, call_next):
        # 访客标识必须在处理函数之前落到上下文里：限额按访客计数，
        # 且演示环境位于 Cloudflare 之后，真实来源 IP 只在请求头上。
        token = quota.identify(quota.client_key(
            request.headers, request.client.host if request.client else '-'))
        try:
            return await _boundary(request, call_next)
        finally:
            quota.release(token)
    async def _boundary(request: Request, call_next):
        # Public, allow-listed release assets are served without the workbench
        # Basic Auth prompt. No directory listing or arbitrary file path is
        # exposed; all other routes keep the normal authentication boundary.
        if request.url.path.startswith('/downloads/'):
            return await call_next(request)
        password = os.getenv('ZHILIAN_ACCESS_PASSWORD', '')
        if password:
            valid = False
            try:
                scheme, token = request.headers.get('authorization', '').split(' ', 1)
                user, supplied = base64.b64decode(token, validate=True).decode().split(':', 1)
                valid = scheme.lower() == 'basic' and hmac.compare_digest(supplied.encode(), password.encode()) and
            user == 'zhilian'
            except (ValueError, UnicodeError):
                pass

            if not valid:
                return JSONResponse({'detail': '请使用用户名 zhilian 与部署密码登录'}, status_code=401,
                                    headers={'WWW-Authenticate': 'Basic realm="Zhilian", charset="UTF-8"'})
        if request.method not in ('GET', 'HEAD', 'OPTIONS'):
            origin = request.headers.get('origin')
            if origin and urlparse(origin).netloc != request.headers.get('host'):
                return JSONResponse({'detail': '拒绝跨站修改请求'}, status_code=403)
        response = await call_next(request)
        response.headers['X-Content-Type-Options'] = 'nosniff'
        response.headers['Referrer-Policy'] = 'same-origin'
        response.headers['X-Frame-Options'] = 'DENY'

```

```

        if request.url.path.startswith('/api/'):
            response.headers['Cache-Control'] = 'no-store'
        return response
@app.exception_handler(ValueError)
async def invalid(request, exc):
    return JSONResponse({'detail': str(exc)}, status_code=400)
@app.exception_handler(FileNotFoundError)
async def missing(request, exc):
    return JSONResponse({'detail': '项目或文件不存在'}, status_code=404)
@app.get('/api/health')
def health():
    return {'status': 'ok', 'version': __version__, 'model': config(),
            'model_mode': mode(), 'model_mode_label': mode_label(),
            'ocr': ocr.config(),
            'local_reranker': reranker.status(),
            'quota': quota.config(),
            'auto_export': os.getenv('ZHILIAN_AUTO_EXPORT', '').strip().lower() in ('1', 'true'),
            'password_protected': bool(os.getenv('ZHILIAN_ACCESS_PASSWORD')),
            'pdf_backends': available_docx_backends()}
@app.get('/downloads/{filename}')
def public_download(filename: str):
    if filename not in PUBLIC_DOWNLOADS:
        raise HTTPException(status_code=404, detail='下载文件不存在')
    path = (DOWNLOAD_ROOT / filename).resolve()
    if DOWNLOAD_ROOT not in path.parents or not path.is_file():
        raise HTTPException(status_code=404, detail='下载文件不存在')
    return FileResponse(path, filename=filename,
                        headers={'Cache-Control': 'public, max-age=86400'})
@app.get('/api/quota')
def quota_state():
    """只读额度查询：演示时可直接展示“还剩多少次模型调用”。"""
    return {**quota.config(), **quota.snapshot()}
@app.get('/api/projects')
def projects():
    return store.list()
@app.post('/api/projects/demo')
def demo(request: Request):
    with tempfile.TemporaryDirectory() as tmp:
        # 线上演示显式请求语义改写夹具：默认 API 保持兼容，便于离线

        # 集成测试和第三方调用继续使用稳定的确定性样例。
        semantic_demo = request.query_params.get('semantic', '').lower() in ('1', 'true', 'yes', 'on')
        return store.create('销售分析 • 演示项目', create_demo(tmp, semantic_demo=semantic_demo), demo=True)
@app.get('/api/template')
def template():
    folder = store.root / '_templates'
    if not (folder / '业务数据.xlsx').exists():
        create_demo(folder)
    return FileResponse(folder / '业务数据.xlsx', filename='知链事实表模板.xlsx')
@app.post('/api/projects')
async def upload(name: str = Form('新项目'), files: list[UploadFile] = File(...)):
    if len(files) > 10:
        raise ValueError('最多上传10份文件')
    paths, total, names = [], 0, set()
    with tempfile.TemporaryDirectory() as tmp:
        try:

```

```

        for f in files:
            filename = (f.filename or '').replace('\\', '/').split('/')[-1]
            if not filename or filename in names or Path(filename).suffix.lower() not in ('.xlsx', '.docx',
'.pptx', *ocr.IMAGE_EXTENSIONS):
                raise ValueError('文件名重复或类型不受支持, 仅接受xlsx、docx、pptx或图片文件(png/jpg/gif/web
p/bmp/tiff)')

            if len(filename) > 150 or any(c in filename for c in '<>:"|?*'):
                raise ValueError('文件名无效或过长')
            names.add(filename)
            path = Path(tmp) / filename
            with path.open('wb') as out:
                while chunk := await f.read(1024 * 1024):
                    total += len(chunk)
                    if total > 30 * 1024 * 1024:
                        raise ValueError('每批上传总大小不超过30MB')
                    out.write(chunk)
            paths.append(path)
        return await run_in_threadpool(store.create, name, paths)
    finally:
        for f in files:
            await f.close()
@app.post('/api/projects/from-pdf')
async def upload_pdf_project(name: str = Form('PDF 导入项目'), subject: str = Form(''), file: UploadFile = File(
...)):
    """Convert one PDF to Word/Excel sources, then create an Office project.
    The original PDF is retained for provenance but is never treated as an
    editable project document. PDFs with no usable structured facts are
    rejected with a message directing the user to review the generated
    intermediate files first.
    """
    filename = (file.filename or '').replace('\\', '/').split('/')[-1]
    if Path(filename).suffix.lower() != '.pdf':
        raise ValueError('请选择 .pdf 文件')
    if len(filename) > 150 or any(c in filename for c in '<>:"|?*'):
        raise ValueError('PDF 文件名无效或过长')
    with tempfile.TemporaryDirectory() as tmp:
        path = Path(tmp) / filename

        total = 0
        try:
            with path.open('wb') as out:
                while chunk := await file.read(1024 * 1024):
                    total += len(chunk)
                    if total > 100 * 1024 * 1024:
                        raise ValueError('PDF 不超过100MB')
                    out.write(chunk)
            return await run_in_threadpool(store.create_from_pdf, name, path, subject.strip() or None)
        finally:
            await file.close()
@app.post('/api/projects/{wid}/documents')
async def append_documents(wid: str, revision: int = Form(...), files: list[UploadFile] = File(...),
                           continue_on_error: bool = Form(False)):
    """Append a bounded batch of Word/PPT/image files to an existing project.
    The Excel source remains the project's single source of truth and is
    deliberately rejected here. Clients can upload dozens of documents by
    repeating this endpoint; each request is committed as one transaction.

```

```

"""
max_batch = int(os.getenv('ZHILIAN_MAX_APPEND_BATCH', '50'))
if len(files) > max_batch:
    raise ValueError(f'单批最多追加{max_batch}份文档，请分批上传')
paths, total, names = [], 0, set()
with tempfile.TemporaryDirectory() as tmp:
    try:
        for f in files:
            filename = (f.filename or '').replace('\\', '/').split('/')[-1]
            suffix = Path(filename).suffix.lower()
            if not filename or filename in names or suffix not in ('.docx', '.pptx', *ocr.IMAGE_EXTENSIONS):
                raise ValueError('追加批次只接受docx、pptx或图片文件，不允许再次上传xlsx')
            if len(filename) > 150 or any(c in filename for c in '<>:"|?*'):
                raise ValueError('文件名无效或过长')
            names.add(filename)
            path = Path(tmp) / filename
            with path.open('wb') as out:
                while chunk := await f.read(1024 * 1024):
                    total += len(chunk)
                    if total > 100 * 1024 * 1024:
                        raise ValueError('每批追加总大小不超过100MB')
                    out.write(chunk)
            paths.append(path)
    if not continue_on_error:
        return await run_in_threadpool(store.append_documents, wid, revision, paths)
    # Tolerant mode commits each file as its own transaction. A
    # corrupt or duplicate document is reported while successful
    # files remain available and can be reviewed immediately.
    current_revision = revision
    accepted, rejected = [], []
    for path in paths:
        try:

            result = await run_in_threadpool(store.append_documents, wid, current_revision, [path])
            current_revision = result['revision']
            accepted.append({'name': path.name, 'batch': result.get('batch')})
        except Exception as exc:
            rejected.append({'name': path.name, 'reason': str(exc) or type(exc).__name__})
    with store.lock:
        state = store.public(store.read(wid))
        state['batch'] = {
            'status': 'partial' if rejected and accepted else ('failed' if rejected else 'done'),
            'count': len(accepted), 'rejected': len(rejected),
            'revision': current_revision,
        }
        state['batch_results'] = {'accepted': accepted, 'rejected': rejected}
    return state
finally:
    for f in files:
        await f.close()
@app.get('/api/projects/{wid}/batches/{batch_id}')
def batch_status(wid: str, batch_id: str):
    return store.batch(wid, batch_id)
@app.get('/api/projects/{wid}')
def get_project(wid: str):
    with store.lock:

```

```

        return store.public(store.read(wid))
@app.delete('/api/projects/{wid}')
def delete_project(wid: str):
    """Delete a project selected from 我的项目 after client confirmation."""
    return store.delete(wid)
@app.get('/api/projects/{wid}/report')
def report(wid: str):
    with store.lock:
        ws = store.read(wid)
        store.verify(ws, ws['revision'])
        return {'revision': ws['revision'], 'report': build_report(ws)}
@app.post('/api/projects/{wid}/links')
def confirm(wid: str, body: LinksRequest):
    return store.confirm(wid, body.revision, [i.model_dump() for i in body.links])
@app.post('/api/projects/{wid}/facts')
def change(wid: str, body: ChangeRequest):
    return store.change(wid, body.revision, body.values)
@app.get('/api/projects/{wid}/derivations')
def list_derivations(wid: str):
    with store.lock:
        ws = store.read(wid)
        derivations = store.derivations_of(ws)
        return {'revision': ws['revision'], 'derivations': derivations.records,
                'order': derivations.order, 'cycles': derivations.cycles,
                'graph': ws.get('graph') or {}}
@app.post('/api/projects/{wid}/derivations')
def set_derivations(wid: str, body: DerivationsRequest):

    return store.set_derivations(wid, body.revision, [i.model_dump() for i in body.derivations])
@app.delete('/api/projects/{wid}/derivations/{rid}')
def remove_derivation(wid: str, rid: str, revision: int):
    return store.remove_derivation(wid, revision, rid)
async def uploaded_source(wid, revision, file, operation):
    try:
        if Path(file.filename or '').suffix.lower() != '.xlsx':
            raise ValueError('请选择保存后的 .xlsx 文件')
        with tempfile.TemporaryDirectory() as tmp:
            path = Path(tmp) / 'updated.xlsx'
            total = 0
            with path.open('wb') as out:
                while chunk := await file.read(1024 * 1024):
                    total += len(chunk)
                    if total > 30 * 1024 * 1024:
                        raise ValueError('更新表不超过30MB')
                    out.write(chunk)
            return await run_in_threadpool(operation, wid, revision, path)
    finally:
        await file.close()
@app.post('/api/projects/{wid}/source/preview')
async def preview_source(wid: str, revision: int = Form(...), file: UploadFile = File(...)):
    return await uploaded_source(wid, revision, file, store.preview_source)
@app.post('/api/projects/{wid}/source')
async def import_source(wid: str, revision: int = Form(...), file: UploadFile = File(...)):
    return await uploaded_source(wid, revision, file, store.import_source)
@app.post('/api/projects/{wid}/repair')
def repair(wid: str, body: RepairRequest):

```

```

        return store.repair(wid, body.revision, body.claim_ids)
@app.post('/api/projects/{wid}/undo')
def undo(wid: str, body: RevisionRequest):
    return store.undo(wid, body.revision)
@app.post('/api/projects/{wid}/suggest')
def suggestions(wid: str, body: RevisionRequest):
    with store.lock:
        ws = store.read(wid)
        store.verify(ws, body.revision)
        selected = mode(ws)
    with using_mode(selected):
        if not remote_allowed():
            raise ValueError(f'当前为 {selected} 模式，不允许调用远程 API')
        eligible = [c for c in ws['claims'] if not c['confirmed'] and c['kind'] != 'chart']
        if eligible and config()['enabled'] and len(ws['facts']) <= 150:
            gate = quota.check()
            if not gate['allowed']:
                raise ValueError(gate['reason'])
            quota.consume()
            result = suggest_links(ws['claims'], ws['facts'])
            call_metrics = consume_last_call_metrics()
    with store.lock:

        ws = store.read(wid)
        store.verify(ws, body.revision)
        ws['suggestions'] = result
        store.append_model_call(ws, call_metrics, node='link_agent', mode_value=selected,
                                candidates_returned=len(result))

        ws['revision'] += 1
        ws['audit'].append({'time': now(), 'event': '模型建议', 'detail': f'模型 {config()["model"]} 提出 {len(result)} 项建议，尚未确认'})
        store.write(ws)
        return store.public(ws)
@app.post('/api/projects/{wid}/diagnosis/explain')
def diagnosis_explain(wid: str, body: RevisionRequest):
    with store.lock:
        ws = store.read(wid)
        store.verify(ws, body.revision)
        records = store.public(ws)['diagnosis']
        selected = mode(ws)
    with using_mode(selected):
        can_call = remote_allowed() and config()['enabled'] and bool(records)
        if can_call:
            gate = quota.check()
            if not gate['allowed']:
                raise ValueError(gate['reason'])
            quota.consume()
            explanations = explain_diagnosis(records)
            call_metrics = consume_last_call_metrics()
    with store.lock:
        ws = store.read(wid)
        store.verify(ws, body.revision)
        ws['diagnosis_explanations'] = explanations
        store.append_model_call(ws, call_metrics, node='diagnosis_agent', mode_value=selected,
                                candidates_returned=len(explanations))

        ws['revision'] += 1

```

```

ws['audit'].append({'time': now(), 'event': '诊断解释',
                    'revision': ws['revision'],
                    'detail': (f'模型 {config()["model"]} 返回 {len(explanations)} 项解释; 未返回的项使用
确定性模板'

                    if can_call
                    else ('未配置 DeepSeek, 使用确定性模板, 未调用模型'
                           if not config()['enabled']
                           else f'当前为 {selected} 模式, 使用确定性模板, 未调用远程模型'))))

store.write(ws)
return store.public(ws)
@app.post('/api/projects/{wid}/ocr/run')
def ocr_run(wid: str, body: OcrRunRequest):
    return store.ocr_run(wid, body.revision, body.image_ids)
@app.post('/api/projects/{wid}/ocr/confirm')
def ocr_confirm(wid: str, body: OcrConfirmRequest):
    return store.ocr_confirm(wid, body.revision, [i.model_dump() for i in body.items])
@app.get('/api/projects/{wid}/images/{iid}')
def image_download(wid: str, iid: str):
    with store.lock:

        ws = store.read(wid)
        store.verify(ws, ws['revision'])
        image = next((i for i in ws.get('images', []) if i['id'] == iid), None)
        if not image:
            raise HTTPException(404, '图片不存在')
        path = store.image_path(ws, image)
        media_type = image['mime'] if image['mime'] in ocr.MIME_EXT else 'application/octet-stream'
        return FileResponse(path, media_type=media_type,
                            headers={'Content-Security-Policy': "default-src 'none'; sandbox"})
@app.get('/api/projects/{wid}/files/{fid}')
def download(wid: str, fid: str):
    ws = store.read(wid)
    store.verify(ws, ws['revision'])
    d = next((d for d in ws['documents'] if d['id'] == fid), None)
    if not d:
        raise HTTPException(404, '文件不存在')
    return FileResponse(store.folder(wid) / ws['generation'] / d['stored_name'], filename=d['name'])
@app.get('/api/projects/{wid}/pdf-origins/{index}')
def download_pdf_origin(wid: str, index: int):
    with store.lock:
        ws = store.read(wid)
        store.verify(ws, ws['revision'])
        origins = ws.get('pdf_origins', [])
        if index < 0 or index >= len(origins):
            raise HTTPException(404, '原始 PDF 不存在')
        origin = origins[index]
        path = store.folder(wid) / ws['generation'] / origin['stored_name']
        if not path.is_file() or digest(path) != origin.get('sha256'):
            raise HTTPException(409, '原始 PDF 校验失败, 请重新导入')
        return FileResponse(path, filename=origin['name'], media_type='application/pdf')
@app.post('/api/projects/{wid}/pdf-revised')
def export_revised_pdf(wid: str, body: RevisionRequest):
    """Generate an independent PDF from the current Word revision."""
    return store.export_revised_pdf(wid, body.revision)
@app.get('/api/projects/{wid}/pdf-revisions/{index}')
def download_revised_pdf(wid: str, index: int):

```

```
with store.lock:
    ws = store.read(wid)
    store.verify(ws, ws['revision'])
    items = ws.get('pdf_revisions', [])
    if index < 0 or index >= len(items):
        raise HTTPException(404, '修订版 PDF 不存在')
    item = items[index]
    verification = item.get('verification') or {}
    if verification.get('status') != 'passed':
        raise HTTPException(409, '修订版 PDF 未通过严格视觉一致性验收, 请先查看转换清单和差异页')
    path = store.folder(wid) / ws['generation'] / item.get('stored_name', '')
    if not path.is_file():
        raise HTTPException(404, '修订版 PDF 不存在')
    return FileResponse(path, filename=item.get('name', path.name), media_type='application/pdf')

@app.get('/api/projects/{wid}/pdf-revisions/{index}/manifest')
def download_revised_manifest(wid: str, index: int):
    with store.lock:
        ws = store.read(wid)
        store.verify(ws, ws['revision'])
        items = ws.get('pdf_revisions', [])
        if index < 0 or index >= len(items):
            raise HTTPException(404, '修订版 PDF 清单不存在')
        item = items[index]
        path = store.folder(wid) / ws['generation'] / item.get('manifest_stored_name', '')
        if not path.is_file():
            raise HTTPException(404, '修订版 PDF 清单不存在')
        return FileResponse(path, filename=path.name, media_type='application/json')

@app.post('/api/projects/{wid}/agent/run')
def agent_run(wid: str, body: RevisionRequest):
    return run_agent(store, wid, body.revision)

@app.post('/api/projects/{wid}/model-mode')
def set_model_mode(wid: str, body: ModelModeRequest):
    return store.set_model_mode(wid, body.revision, body.mode)

@app.post('/api/projects/{wid}/agent/decide')
def agent_decide(wid: str, body: AgentDecisionRequest):
    return decide(store, wid, body.revision, body.decisions)

@app.get('/api/projects/{wid}/agent/status')
def get_agent_status(wid: str):
    return agent_status(store, wid)

@app.post('/api/projects/{wid}/review/cross-document')
def cross_document_review(wid: str, body: RevisionRequest):
    return run_cross_document_audit(store, wid, body.revision)

@app.post('/api/projects/{wid}/review/repair-plan')
def repair_plan(wid: str, body: RevisionRequest):
    return run_repair_plan(store, wid, body.revision)

@app.get('/api/projects/{wid}/export')
def export(wid: str):
    return FileResponse(store.archive(wid), filename='知链成果与核验记录.zip', media_type='application/zip')

@app.post('/api/projects/{wid}/export-local')
def export_local(wid: str, body: RevisionRequest):
    with store.lock:
        ws = store.read(wid)
        store.verify(ws, body.revision)
    return store.export_local(wid)

@app.get('/')

```

```

def index():
    return FileResponse(BASE / 'web' / 'index.html')
app.mount('/static', StaticFiles(directory=BASE / 'web'), name='static')
return app
app = create_app()
// ===== 文件: zhilian/engine.py =====
"""Deterministic, typed claim checks. No eval, generated code or model arithmetic."""
from __future__ import annotations
import hashlib

import math
import re
from decimal import Decimal, InvalidOperation, ROUND_HALF_UP
KINDS = {'quote': '数值引用', 'growth': '增长率', 'ranking': '排名', 'threshold': '阈值', 'chart': '图表'}
EXTRACTION_VERSION = 7
UNITS = {'元': ('currency', Decimal(1)), '万元': ('currency', Decimal(10000)),
         '亿元': ('currency', Decimal(100000000)), '件': ('count', Decimal(1)),
         '人': ('people', Decimal(1)), 'CNY': ('currency', Decimal(1)),
         '%': ('percentage', Decimal(1))}
# 允许千分位逗号: 真实年报普遍写作 3,379.37 万元, 旧规则只认连续数字。
NUM = r'-?\d{1,3}(?:,\d{3})+(?:\.\d+)?|-?\d+(?:\.\d+)?'
UNIT = r'亿元|万元|元|件|人|CNY|%'
def number(value) -> Decimal:
    if isinstance(value, bool):
        raise ValueError('布尔值不能作为数值')
    text = str(value).strip()
    # 去千分位。仅当逗号确实按三位分组时才剥离, 避免误改 "1,23" 这类异常输入。
    if re.fullmatch(r'-?\d{1,3}(?:,\d{3})+(?:\.\d+)?', text):
        text = text.replace(',', '')
    try:
        n = Decimal(text)
    except (InvalidOperation, ValueError):
        raise ValueError('请输入有效数字')
    if not n.is_finite() or abs(n) > Decimal('1e15'):
        raise ValueError('数字必须有限且绝对值不超过 10^15')
    return n
def fmt(value) -> str:
    n = number(value).quantize(Decimal('.01'), rounding=ROUND_HALF_UP)
    return format(n, 'f').rstrip('0').rstrip('.') if '.' in format(n, 'f') else str(n)
def convert(value, unit, target):
    if unit == target:
        return number(value)
    a, b = UNITS.get(unit), UNITS.get(target)
    if not a or not b or a[0] != b[0]:
        raise ValueError(f'单位不兼容: {unit} 与 {target}')
    return number(value) * a[1] / b[1]
class CheckError(ValueError):
    """带分类码的核验失败。
    旧实现靠比对错误文案字符串来判断分类, 任何一处提示文案被润色
    (例如"请"改成"您")都会让分类静默退化为 needs_review, 进而影响
    前端诊断分组与报告分组。分类改走结构化字段后不再与文案耦合。
    """
    def __init__(self, code, message):
        super().__init__(message)
        self.code = code
def stable_id(*parts):

```

```

    return hashlib.sha256(' '.join(map(str, parts)).encode()).hexdigest()[:18]
# 主体为这些值时不参与定位：它们描述的是“整体”，任何句子都可能命中。
GENERIC_SUBJECTS = ('总计', '整体', '全部')
# 期间是封闭词表，同义写法属于确定性规则该覆盖的范围：真实文档写“报告期/本年/

# 去年同期”，而事实表只写“本期/上期”。不归一化会让上期与本期同分并列，一条普通
# 数值引用被判成“口径不唯一”，甚至让阈值/增长率拿不到唯一来源。
PERIOD_SYNONYMS = {
    '本期': ('本期', '本报告期', '报告期内', '报告期', '本年度', '本年', '当期', '今年'),
    '上期': ('上期', '上年度同期', '上年同期', '去年同期', '上年度', '上年', '去年'),
}
# 只扩充定义相同的名称。营业总收入、主营收入、现金等价物、研发投入
# 均有独立会计口径，不能因词面相似合并；这些仍交给语义候选与用户确认。
METRIC_ALIASES_BY_CANONICAL = {
    '营业收入': ('营业收入', '营收'),
    '资产总计': ('资产总计', '总资产'),
    '总资产': ('资产总计', '总资产'),
    '负债总计': ('负债总计', '总负债'),
    '总负债': ('负债总计', '总负债'),
    '员工人数': ('员工人数', '员工总数'),
    '员工总数': ('员工人数', '员工总数'),
}
def metric_aliases(metric):
    """Return the closed lexical alias set for one fact metric."""
    metric = str(metric or '')
    if metric in METRIC_ALIASES_BY_CANONICAL:
        return METRIC_ALIASES_BY_CANONICAL[metric]
    return (metric,) if metric else ()
def period_in_text(period, text):
    """事实的期间是否在文本中以任意同义写法出现."""
    return any(word in text for word in PERIOD_SYNONYMS.get(period, (period,)) if word)
def _descriptors(fact):
    """一条事实的**定位**描述符：指标与非泛指主体。
    期间刻意不放进这里。旧实现中期间只参与加分，不参与“能否匹配”；
    把它列为必需项会让「A产品销量最高」这类没写期间的句子匹配不到任何事实。
    需要按期间收窄时由 best_facts(period=...) 显式过滤。
    """
    keys = []
    if fact.get('metric'):
        keys.append(fact['metric'])
    if fact.get('subject') and fact['subject'] not in GENERIC_SUBJECTS:
        keys.append(fact['subject'])
    return keys
def build_index(facts):
    """描述符 → 事实ID 的反向索引。
    原实现每段文本都遍历全部事实做子串包含判断，一段的开销是
    O(事实数 × 描述符数)。索引把“哪些事实可能相关”提前算好，
    长文档导入时每段只需检查命中描述符所涉及的事实。
    """
    token_ids = {}
    for fact in facts:
        for key in _descriptors(fact):
            token_ids.setdefault(key, set()).add(fact['id'])
        for alias in metric_aliases(fact.get('metric')):
            token_ids.setdefault(alias, set()).add(fact['id'])

```

```

    return {'token_ids': token_ids}
_INDEX_CACHE = {}
_INDEX_CACHE_MAX = 8
def facts_index(facts, bypass_cache=False):
    """按描述符构造（并可复用）反向索引。改数值不影响索引，描述符变化时自动重建。
    键是指纹，因此命中条件是“这张事实表的指纹已在缓存里”。旧实现写成
    `entry['size'] != len(facts)`——指纹相同则事实条数必然相同，这个比较永远为
    假，真正的失效场景（同条数不同描述符）反而漏网；同时 `> 8` 的整表清空让
    连续处理 9 张不同事实表后缓存归零，跨请求基本零命中。
    """
    if bypass_cache:
        return build_index(facts)
    fingerprint = hash(tuple(sorted(
        (f['id'], f.get('metric', ''), f.get('subject', ''), f.get('period', '')) for f in facts)))
    entry = _INDEX_CACHE.get(fingerprint)
    if entry is None:
        entry = build_index(facts)
        if len(_INDEX_CACHE) >= _INDEX_CACHE_MAX:
            _INDEX_CACHE.clear()
        _INDEX_CACHE[fingerprint] = entry
    return entry
def best_facts(text, facts, *, period=None, index=None):
    """找出文本对应的唯一事实来源。
    语义与旧实现一致：指标、非泛指主体都必须出现在文本中（至少含指标），
    期间出现时额外加分；period 是额外的显式过滤条件。多条并列时返回全部
    候选，由上层判为“口径不唯一”并转人工确认。
    """
    if not text:
        return []
    index = index or build_index(facts)
    token_ids = index['token_ids']
    hit_tokens = [token for token in token_ids if token in text]
    if hit_tokens:
        wanted = set()
        for token in hit_tokens:
            wanted |= token_ids[token]
        candidates = [f for f in facts if f['id'] in wanted]
    else:
        candidates = facts
    scored = []
    for fact in candidates:
        keys = _descriptors(fact)
        if not keys or not any(alias in text for alias in metric_aliases(fact.get('metric'))):
            continue
        if fact.get('subject') and fact['subject'] not in GENERIC_SUBJECTS and fact['subject'] not in text:
            continue
        if period and fact['period'] != period:
            continue
        score = 5
        if fact.get('subject', '') not in GENERIC_SUBJECTS:
            score += 4
        if fact['period'] and period_in_text(fact['period'], text):
            score += 3
        scored.append((score, fact))

```

```

if not scored:
    return []
top = max(s for s, _ in scored)
return [f for s, f in scored if s == top]

def ranking_cohort(metric, facts):
    """排名称谓的可比集合：同指标、同期间的同一口径组。
    排名只有在比较集合完整时才有意义。集合不足两条时必须把“待补齐”暴露给用户，而不是把单条事实当成“最高”的依据；因此抽取和人工候选共用这一处逻辑，避免两边各写一份判断。
    """
    if not metric:
        return []
    return [f['id'] for f in facts if f['metric'] == metric and f['period'] == '本期']

def growth_pair(metric, facts):
    """增长率所需的同口径上期/本期配对；只有两侧各唯一时才成立。
    与 ranking_cohort 同理：增长率论断的证据是一对事实，不是一条。当文本只提供了其中一侧的线索时，用它的指标在事实表里补另一侧，避免把“无法确定”直接推成无候选的死结——人工需要看到一个可勾选的配对。
    """
    if not metric:
        return []
    prev = [f['id'] for f in facts if f['metric'] == metric and f['period'] == '上期']
    curr = [f['id'] for f in facts if f['metric'] == metric and f['period'] == '本期']
    return [prev[0], curr[0]] if len(prev) == 1 and len(curr) == 1 else []

def growth_pairs(facts):
    """事实表里所有“上期+本期”齐全指标配对，按指标去重后返回。
    增长率句子常常只写「较上期增长25%」，上下文里也没有指标名，此时规则无法确定它指哪个指标。与其给出零候选（界面上无任何可勾选选项、decide 直接报错），不如把事实表里所有成对的指标作为候选交给人工，每条都带确定性核验结论。
    """
    pairs = []
    for metric in dict.fromkeys(f.get('metric') for f in facts):
        pair = growth_pair(metric, facts)
        if pair:
            pairs.append((metric, pair))
    return pairs

def extract_claims(block, facts, index=None):
    """Extract separate, non-overlapping assertions, retaining their exact text anchors."""
    result = []
    context = ''
    # 每段只构造/复用一次索引，避免对每条事实重复扫描文本。
    index = index or facts_index(facts)
    patterns = [
        # 真实年报用「较上年/同比/比上年」，且动词常用「增加/减少」而非「增长/下降」。
        # 旧规则只认「较上期|环比」+「增长|下降|持平」，导致年报语料识别率为 0。
        # 年报有两种稳定写法：带比较期间（同比增长25%），以及在同一
        # 句子中直接写“产量增加179.79%”。后者只有在动词后紧跟百分比
        # 时才启用，避免把普通叙述中的“增加”误识别成增长率论断。
        ('growth', r'(?:(?:较上期|环比|较上年|比上年|同比)\s*|'
            r'(?:(?:增长|下降|持平|增加|减少|上升)(?!率)\s*(?:' + NUM + r')\s*%)')
            r'(增长|下降|持平|增加|减少|上升)(?!率)(?:\s*(? + NUM + r')\s*%)?(?!s*d)'),
        ('ranking', r'(.+?)(销售额|销量|收入|支出|得分)(?:并列)?最高'),
        ('threshold', r'(未超过|不超过|超过|不少于|低于|高于)\s*(? + NUM + r')\s*(? + UNIT + r)'),
        ('budget', r'支出(未超过|不超过|超过)预算'),
        # 年报中的数值引用连接词远多于“为/是/达到”。连接词允许和数字之间
    ]

```

```

# 有少量空白；末尾的“无连接词”分支覆盖“营业收入125万元”这类
# 标题式写法。阈值模式排在前面，因此“超过120万元”仍优先识别为阈值。
('quote', r'(?:(?:约为|实现了|录得了|增至|高达|实现|录得|共计|合计|完成|累计|近|为|是|达到|达|is)\s*'
          r'(?=(?:' + NUM + r')\s*(?:亿元|万元|元|件|人|CNY)(?!\\w))('
          + NUM + r')\s*(' + UNIT + r'))',
]
candidate_matches = []
sentence_meta = []
for sentence_index, sentence in enumerate(re.finditer(r'[^! ? ; ;\n]+', block['text'])):
    sentence_meta.append(sentence.group())
    matches = sorted((m.start(), m.end(), kind, m)
                     for kind, pattern in patterns for m in re.finditer(pattern, sentence.group()))
    previous_end = 0
    for marker_start, marker_end, kind, match in matches:
        if marker_start < previous_end:
            continue
        prefix = sentence.group()[previous_end:marker_start]
        # A comma or connector belongs to neither adjacent assertion. Text after
        # the final predicate remains uncovered instead of inheriting its status.
        offset = max(prefix.rfind(', '), prefix.rfind(',')) + 1
        leading = prefix[offset:]
        trim = re.match(r'^[\s,]*(?:(?:并且|而且|同时|以及|且|并)[\s,]*)?', leading).end()
        local_start = previous_end + offset + trim
        start = sentence.start() + local_start
        end = sentence.start() + marker_end
        text = block['text'][start:end]
        leading_punctuation = len(text) - len(text.lstrip(' \t, , '))
        if leading_punctuation:
            start += leading_punctuation
            text = text[leading_punctuation:]
        local_match = re.search(dict(patterns)[kind], text)
        candidate_matches.append((sentence_index, start, end, text, kind, local_match))
        previous_end = marker_end

# The old extractor produced one claim per sentence. Preserve those IDs when
# the old primary kind is still present, while assigning deterministic IDs to
# additional assertions introduced by compound extraction.
legacy_kind, legacy_id_index = {}, {}
legacy_counter = 0
for sentence_index, sentence_text in enumerate(sentence_meta):
    if re.search(patterns[0][1], sentence_text):
        legacy_kind[sentence_index] = 'growth'

    elif re.search(patterns[1][1], sentence_text):
        legacy_kind[sentence_index] = 'ranking'
    elif re.search(patterns[2][1], sentence_text):
        legacy_kind[sentence_index] = 'threshold'
    elif re.search(patterns[3][1], sentence_text):
        legacy_kind[sentence_index] = 'budget'
    elif re.search(patterns[4][1], sentence_text):
        legacy_kind[sentence_index] = 'quote'
    if legacy_kind.get(sentence_index):
        legacy_id_index[sentence_index] = legacy_counter
        legacy_counter += 1

compound_counter, legacy_assigned = 0, set()
# 同一段文本在一次抽取里会被反复求来源：算 source_text 时一次、按期间各一次、
# 每个论断分支又一次、循环末尾还有一次。best_facts 每次都要遍历候选事实做

```

```

# 子串判断，于是单段的开销按论断数线性放大。这里按（文本，期间）记忆本轮结果——
# 事实与索引在函数内是常量，记忆化不改变语义，只去掉重复计算。
resolved = {}
def resolve(text, period=None):
    key = (text, period)
    hit = resolved.get(key)
    if hit is None:
        hit = best_facts(text, facts, period=period, index=index)
        resolved[key] = hit
    return hit
for sentence_index, start, end, text, marker_kind, match in candidate_matches:
    refs, spec, kind, issue = [], {}, None, ''
    # Only an elided predicate can inherit a metric from preceding context.
    # A newly named, uncovered metric must not borrow the preceding source.
    source_text = text if resolve(text) or match.start() > 0 else context
    if marker_kind == 'growth':
        growth = match
        kind = 'growth'
        comparison_explicit = bool(re.match(r'(?<较上期|环比|较上年|比上年|同比)', growth.group()))
        curr = resolve(source_text, '本期')
        prev = resolve(source_text, '上期')
        suggested = []
        if len(curr) == len(prev) == 1:
            refs = [prev[0]['id'], curr[0]['id']]
        else:
            issue = '未唯一确定同口径的上期与本期数据，请确认关联'
            # 只要有一侧唯一，就能用它的指标把另一侧补出来，作为人工候选；
            # 否则这条论断在界面上会变成“零选项”的死结（无法勾选、无法确认）。
            known = curr or prev
            if len(known) == 1:
                suggested = growth_pair(known[0]['metric'], facts)
            # 方向词扩展：年报用「增加/减少/上升」，旧字典只有「增长/下降/持平」。
            # 同时用 number() 而非 float()，以支持千分位数值。
            direction_word = {'增长': 1, '增加': 1, '上升': 1,
                              '下降': -1, '减少': -1, '持平': 0}[growth[1]]
            spec = {'span': [growth.start(1), growth.end()],

                    'reported': float(number(growth[2])) if growth[2] else 0.0,
                    'qualitative': growth[2] is None, 'direction': direction_word}
        if not comparison_explicit:
            # “增加25%”没有明说相对哪一期，不能静默假设上期。
            # 保留抽取锚点，但必须由用户补全比较基准与两侧事实。
            spec['comparison_period_explicit'] = False
            refs = []
            issue = '增长率的比较基准未明确，请确认同口径的两期来源'
        if suggested:
            spec['suggested_refs'] = suggested
        if direction_word < 0:
            spec['reported'] = -abs(spec['reported'])
    elif marker_kind == 'ranking':
        rank = match
        kind = 'ranking'
        metric = rank[2]
        refs = ranking_cohort(metric, facts)
        spec = {'metric': metric, 'winners': rank[1].strip().split(',') , 'span': [rank.start(), rank.end()]}
        issue = '' if len(refs) >= 2 else '排名至少需要两个可比较对象'

```

```

elif marker_kind == 'threshold':
    threshold = match
    kind = 'threshold'
    fs = resolve(source_text)
    refs = [fs[0]['id']] if len(fs) == 1 else []
    positive, negative, op = ('超过', '未超过', '>')
    if threshold[1] in ('不少于', '低于'):
        positive, negative, op = ('不少于', '低于', '>=')
    spec = {'limit': float(number(threshold[2])), 'unit': threshold[3], 'positive': positive,
           'negative': negative, 'op': op, 'reported_positive': threshold[1] == positive,
           'span': [threshold.start(1), threshold.end(1)]}
elif marker_kind == 'budget':
    kind = 'threshold'
    # Resolve each side from an explicit metric phrase; both period and
    # metric are required so similarly named facts stay ambiguous.
    fs1, fs2 = resolve('本期支出'), resolve('本期预算')
    refs = [fs1[0]['id'], fs2[0]['id']] if len(fs1) == len(fs2) == 1 else []
    m = match
    spec = {'positive': '超过', 'negative': '未超过', 'op': '>', 'reported_positive': m[1] == '超过', 'span':
: [m.start(), m.end()]}
    spec['span'] = [m.start(1), m.end(1)]
elif marker_kind == 'quote':
    quote = match
    kind = 'quote'
    fs = resolve(text)
    refs = [fs[0]['id']] if len(fs) == 1 else []
    spec = {'reported': float(number(quote[1])), 'unit': quote[2], 'span': [quote.start(1), quote.end(1)]}
if kind:
    if not refs:
        issue = issue or '未找到唯一来源, 需手动关联事实'
    if legacy_kind.get(sentence_index) == kind and sentence_index not in legacy_assigned:
        legacy_assigned.add(sentence_index)

    source_key = stable_id(block['file_id'], block['location'], kind, legacy_id_index[sentence_index])
else:
    source_key = stable_id(block['file_id'], block['location'], 'compound-v2', compound_counter, kind)
    compound_counter += 1
    result.append({'id': source_key,
                  'file_id': block['file_id'], 'location': block['location'], 'label': block['label'],
                  'original': text, 'start': start, 'end': end,
                  'kind': kind, 'refs': refs, 'spec': spec, 'confirmed': False,
                  'extraction': '规则识别', 'issue': issue})
    context = text if resolve(text) else context
return result
def unmatched_spans(blocks, claims):
    """Return uncovered prose, including gaps in otherwise recognized paragraphs."""
    result = []
    for block in blocks:
        ranges = sorted((c['start'], c['end']) for c in claims if c['kind'] != 'chart'
                        and c['file_id'] == block['file_id'] and c['location'] == block['location'])
        offset = 0
        for start, end in ranges + [(len(block['text']), len(block['text']))]:
            raw = block['text'][offset:start]
            text = raw.strip(' \t\r\n。 ! ? ; , , , ')
            if text and not re.fullmatch(r'(?:(?:并且|而且|同时|以及|且|并)', text):
                leading = len(raw) - len(raw.lstrip(' \t\r\n。 ! ? ; , , , '))

```

```

        result.append(dict(block, text=text, start=offset + leading, end=start))
    offset = max(offset, end)
    return result
def replace_span(text, span, replacement):
    return text[:span[0]] + replacement + text[span[1]:]
def check(claim, facts):
    byid = {f['id']: f for f in facts}
    out = {'claim_id': claim['id'], 'status': 'unverifiable', 'code': 'needs_review', 'expected': None, 'reason': ''
, 'evidence': []}
    try:
        refs = [byid[i] for i in claim['refs']]
        out['evidence'] = [{k: f.get(k) for k in ('id', 'subject', 'metric', 'period', 'value', 'unit', 'scope', 'sheet', 'cell')} for f in refs]
        if not refs or any(f.get('value') is None for f in refs):
            raise CheckError('missing_data', '来源缺失或数值不可计算')
        k, s, old = claim['kind'], claim['spec'], claim['original']
        expected, ok, calculation = old, False, ''
        if k == 'quote':
            if len(refs) != 1:
                raise CheckError(None, '数值引用必须关联一个事实')
            value = convert(refs[0]['value'], refs[0]['unit'], s['unit'])
            raw = old[s['span'][0]:s['span'][1]]
            numeric_span = bool(re.fullmatch(r'?:' + NUM + r')', raw))
            decimals = len(raw.split('.', 1)[1]) if numeric_span and '.' in raw else 0 if numeric_span else 2
            precision = Decimal(1).scaleb(-decimals)
            displayed = value.quantize(precision, rounding=ROUND_HALF_UP)
            reported = number(raw) if numeric_span else number(s['reported'])
            ok = displayed == reported
            replacement = format(displayed, 'f')

            expected = replace_span(old, s['span'], replacement)
            calculation = f"按原文 {decimals} 位小数四舍五入: {refs[0]['value']} {refs[0]['unit']} = {replacement} {s['unit']}"
        elif k == 'growth':
            if len(refs) != 2:
                raise CheckError(None, '增长率需要上期、本期两个事实')
            a, b = refs
            if any(a[x] != b[x] for x in ('subject', 'metric', 'scope')) or a['period'] != '上期' or b['period'] != '本期':
                raise CheckError('scope_mismatch', '增长率的主体、指标、口径或上期本期顺序不一致')
            before, after = number(a['value']), convert(b['value'], b['unit'], a['unit'])
            if before <= 0:
                raise CheckError('needs_review', '上期数值必须大于零; 零值或负基期转人工复核')
            rate = (after - before) / before * 100
            rounded = rate.quantize(Decimal('.01'), rounding=ROUND_HALF_UP)
            if s.get('qualitative'):
                direction = 1 if rate > 0 else -1 if rate < 0 else 0
                ok = direction == s['direction']
                wording = {1: '增长', -1: '下降', 0: '持平'}[direction]
            else:
                ok = rounded == number(s['reported']).quantize(Decimal('.01'), rounding=ROUND_HALF_UP)
                wording = '持平' if rounded == 0 else ('增长' if rounded > 0 else '下降') + fmt(abs(rounded)) + '%'
            expected = replace_span(old, s['span'], wording)
            calculation = f"({fmt(after)} {fmt(before)}) / {fmt(before)} × 100 = {fmt(rate)}%"
        elif k == 'ranking':
            if len(refs) < 2:

```

```

        raise CheckError('source_conflict', '排名至少需要两个对象')
    first = refs[0]
    if any(any(f[x] != first[x] for x in ('metric', 'period', 'scope')) for f in refs):
        raise CheckError('scope_mismatch', '排名对象的指标、期间或统计口径不一致')
    if len({f['subject'] for f in refs}) != len(refs):
        raise CheckError('source_conflict', '排名对象重复, 请核对比较集合')
    values = [convert(f['value'], f['unit'], first['unit']) for f in refs]
    winners = [f['subject'] for f, v in zip(refs, values) if v == max(values)]
    ok = set(winners) == set(s['winners'])
    target = ', '.join(winners) + s['metric'] + ('并列最高' if len(winners) > 1 else '最高')
    expected = replace_span(old, s['span'], target)
    calculation = '; '.join(f"{f['subject']} {fmt(v)} {first['unit']}" for f, v in zip(refs, values)) + ' (仅
针对已确认的比较集合)'
    elif k == 'threshold':
        if 'limit' in s:
            if len(refs) != 1:
                raise CheckError(None, '固定阈值需要一个事实')
            value, limit, unit = convert(refs[0]['value'], refs[0]['unit'], s['unit']), number(s['limit']), s['u
nit']
        else:
            if len(refs) != 2:
                raise CheckError(None, '支出与预算需要两个事实')
            a, b = refs
            if any(a[x] != b[x] for x in ('subject', 'period', 'scope')) or a['metric'] != '支出' or b['metric']
!= '预算':
                raise CheckError('scope_mismatch', '支出与预算的主体、期间、口径或指标不一致')
            value, limit, unit = number(a['value']), convert(b['value'], b['unit'], a['unit']), a['unit']
            truth = value > limit if s['op'] == '>' else value >= limit
            ok = truth == s['reported_positive']

    expected = replace_span(old, s['span'], s['positive'] if truth else s['negative'])
    calculation = f"{fmt(value)}{unit} {s['op']} {fmt(limit)}{unit} → {'成立' if truth else '不成立'}"
    elif k == 'chart':
        if len(refs) != len(s['values']):
            raise CheckError('source_conflict', '图表关联数量与原始类别不一致')
        values = [convert(f['value'], f['unit'], s['unit']) for f in refs]
        if any(f['metric'] != refs[0]['metric'] or f['scope'] != refs[0]['scope'] or f['subject'] != refs[0]['su
bject'] for f in refs):
            raise CheckError('scope_mismatch', '图表系列的指标、主体或口径不一致')
        if [f['period'] for f in refs] != s['categories']:
            raise CheckError('scope_mismatch', '图表类别与来源期间不匹配')
        ok = all(number(x) == y for x, y in zip(s['values'], values))
        expected = s['series'] + ': ' + ', '.join(f"{c} {fmt(v)}{s['unit']}" for c, v in zip(s['categories'], va
lues))
        calculation = '按已确认的系列与类别重建原生图表数据'
    else:
        raise CheckError('needs_review', '不支持的论断类型')
    out.update(status='consistent' if ok else 'inconsistent',
               code='consistent' if ok else 'inconsistent',
               expected=old if ok else expected, reason=calculation)
except (ValueError, KeyError, IndexError, InvalidOperation, TypeError) as exc:
    reason = str(exc)
    out['reason'] = reason
    # 分类取结构化错误码, 不再比对提示文案。兜底为 missing_data 的理由:
    # 本项目里核验无法进行, 主要就是“该有的来源没有或不可计算”;
    # 多给了来源 (引用数量超出该论断所需) 才单独归为 source_count。

```

```

# 引用指向不存在的事实时同样归为 missing_data——那是最根本的缺失。
code = getattr(exc, 'code', None)
if code is None:
    code = 'missing_data'
    if claim['kind'] == 'quote':
        code = 'source_count' if len(claim['refs']) > 1 else 'missing_data'
    elif claim['kind'] == 'threshold' and 'limit' in claim.get('spec', {}):
        code = 'source_count' if len(claim['refs']) > 1 else 'missing_data'
    elif claim['kind'] in ('growth', 'chart'):
        expected = 2 if claim['kind'] == 'growth' else len(claim.get('spec', {}).get('values', []))
        code = 'source_count' if len(claim['refs']) > expected else 'missing_data'
if any(i not in byid for i in claim['refs']):
    code = 'missing_data'
out['code'] = code
return out
def inspect(workspace):
    checks = [dict(check(c, workspace['facts']), confirmed=c['confirmed']) for c in workspace['claims']]
    summary = {'claims': len(checks), 'consistent': 0, 'inconsistent': 0, 'unverifiable': 0,
               'pending': 0, 'repairable': 0, 'documents': len(workspace['documents']), 'facts': len(workspace['facts'])}
    for r in checks:
        summary[r['status']] += 1
        summary['pending'] += not r['confirmed']
        summary['repairable'] += r['confirmed'] and r['status'] == 'inconsistent'
    return checks, summary
// ===== 文件: zhilian/store.py =====
"""Local persistent workspaces and transactional versions."""

from __future__ import annotations
import copy
import json
import os
import re
import shutil
import tempfile
import threading
import uuid
from contextlib import contextmanager
from datetime import datetime, timezone
from decimal import Decimal
from pathlib import Path
from zipfile import ZipFile, ZIP_DEFLATED
from .engine import extract_claims, facts_index, inspect, check, number, unmatched_spans
from .graph import Derivations, compute, impact
from .diagnose import diagnose
from .report import build_report
from .office import (read_facts, inspect_fact_source, read_document, read_images,
                     digest, apply_document, update_workbook)
from .pdf_office_export import export_pdf_to_office
from .pdf_revised_export import (build_revised_pdf_manifest, compare_pdf_visual_fidelity,
                                 export_docx_to_pdf, verify_revised_pdf)

from . import ocr, llm, quota
from . import review
def now():
    return datetime.now(timezone.utc).isoformat()
class Store:

```

```

def __init__(self, root):
    self.root = Path(root)
    self.root.mkdir(parents=True, exist_ok=True)
    self.lock = threading.RLock()
def folder(self, wid):
    if not re.fullmatch(r'[a-f0-9]{32}', wid):
        raise ValueError('无效的项目ID')
    return self.root / wid
def read(self, wid):
    path = self.folder(wid) / 'state.json'
    if not path.is_file():
        raise FileNotFoundError('项目不存在')
    return json.loads(path.read_text(encoding='utf-8'))
def write(self, ws):
    folder = self.folder(ws['id'])
    tmp = folder / 'state.tmp'
    tmp.write_text(json.dumps(ws, ensure_ascii=False, indent=2, allow_nan=False), encoding='utf-8')
    os.replace(tmp, folder / 'state.json')
@staticmethod
def append_model_call(ws, metrics, *, node, mode_value=None, candidates_returned=0):
    """Attach a secret-free model ledger entry to a workspace."""
    if not metrics:
        return
    entry = copy.deepcopy(metrics)
    entry['node'] = node
    if mode_value:
        entry['mode'] = mode_value
    entry['candidates_returned'] = int(candidates_returned or 0)
    entry['at'] = now()
    ws.setdefault('model_calls', []).append(entry)
@staticmethod
def invalidate_reviews(ws, reason='工作区内容已变化, 请重新运行审计'):
    """Mark review snapshots stale whenever evidence or claims change."""
    for key in ('cross_audit', 'repair_plan'):
        snapshot = ws.get(key)
        if isinstance(snapshot, dict):
            snapshot['stale'] = True
            snapshot['stale_reason'] = reason
def list(self):
    out = []
    for path in self.root.glob('*/*state.json'):
        try:
            ws = json.loads(path.read_text(encoding='utf-8'))
            out.append({k: ws[k] for k in ('id', 'name', 'created_at', 'revision')})
        except (ValueError, KeyError):
            continue
    return sorted(out, key=lambda x: x['created_at'], reverse=True)
def delete(self, wid):
    """Permanently remove one workspace and its generated file versions."""
    with self.lock:
        folder = self.folder(wid)
        state = folder / 'state.json'
        if not state.is_file():
            raise FileNotFoundError('项目不存在')
        # folder() accepts only a full project UUID, so this cannot resolve

```

```

        # to the data root. Refuse symlinks as an extra containment guard.
        if folder.is_symlink():
            raise ValueError('项目目录无效，拒绝删除')
        name = json.loads(state.read_text(encoding='utf-8')).get('name', '未命名项目')
        shutil.rmtree(folder)
        return {'id': wid, 'name': name, 'deleted': True}

def cross_document_audit(self, wid, revision):
    """Run the read-only cross-document audit and persist its evidence snapshot."""
    with self.lock:
        ws = self.read(wid)
        self.verify(ws, revision)
        result = review.audit_documents(ws)
        semantic_mode = llm.mode(ws)
        if semantic_mode in {'hybrid', 'api'} and llm.config()['enabled'] and result.get('semantic_groups'):
            semantic_outcome = 'succeeded'
            gate = quota.check()
            if not gate['allowed']:

                result['semantic_status'] = 'quota_blocked'
                result['semantic_claims_sent'] = 0
                ws['audit'].append({'time': now(), 'event': '模型额度用尽',
                                    'detail': f'跨文档语义审计未执行: {gate["reason"]}'})
            else:
                quota.consume()
                try:
                    with llm.using_mode(semantic_mode):
                        proposals, sent = review.semantic_findings(ws, result)
                        review.add_semantic_findings(ws, result, proposals)
                        result['semantic_status'] = 'completed'
                        result['semantic_claims_sent'] = sent
                except Exception:
                    semantic_outcome = 'failed'
                    result['semantic_status'] = 'failed'
                    result['semantic_claims_sent'] = sum(len(g.get('claims', [])) for g in result.get('semantic_
groups', []))

                ws['audit'].append({'time': now(), 'event': '跨文档语义审计降级',
                                    'detail': '模型不可用；保留确定性审计结果'})
                metrics = llm.consume_last_call_metrics() or {
                    'provider': llm.config()['provider'], 'model': llm.config()['model'],
                    'duration_ms': 0, 'outcome': semantic_outcome,
                    'prompt_tokens': 0, 'completion_tokens': 0, 'total_tokens': 0,
                    'estimated_cost_cny': None,
                }
                self.append_model_call(ws, metrics, node='audit_agent', mode_value=semantic_mode,
                                      candidates_returned=sum(1 for f in result['findings'] if f.get('source')
== 'model'))
            elif not result.get('semantic_groups'):
                result['semantic_status'] = 'no_cross_file_groups'
                result['created_at'] = now()
                ws['cross_audit'] = result
                ws['audit'].append({'time': now(), 'event': '跨文档审计',
                                    'detail': f'归并 {len(result["groups"])} 组；发现 {len(result["findings"])} 项疑点；
仅生成只读证据'})
                self.write(ws)
            return self.public(ws)

def build_repair_plan(self, wid, revision):

```

```

"""Build a structured, approval-gated plan without changing Office files."""
with self.lock:
    ws = self.read(wid)
    self.verify(ws, revision)
    plan = review.repair_plan(ws)
    plan['agent'] = 'repair_planning_agent'
    plan['created_at'] = now()
    ws['repair_plan'] = plan
    ws['audit'].append({'time': now(), 'event': '生成修复计划',
                        'detail': f'{plan["summary"]["total"]} 项; 高风险 {plan["summary"]["high_risk"]} 项
; 等待用户批准'})
    self.write(ws)
    return self.public(ws)
def scan(self, ws, folder, old_claims=()):
    source = next(d for d in ws['documents'] if d['kind'] == 'xlsx')
    # 基础事实与完整事实分开保存: Excel 只承载基础事实, 派生事实由表达式

    # 定义, 既不能被写回 Excel, 也不能被直接改数值。
    source_path = folder / source['stored_name']
    ws['fact_quality'] = inspect_fact_source(source_path)
    base = read_facts(source_path, source['id'], allow_empty=bool(ws.get('allow_empty_facts')))
    ws['source_facts'] = base
    previous_values = ws.get('graph_values')
    ws['facts'] = self.build_facts(base, ws, previous=previous_values)
    claims, blocks, old = [], [], {c['id']: c for c in old_claims}
    # 事实定位索引一次构造、全程复用, 避免每个正文块重新构造。
    index = facts_index(ws['facts'])
    for d in ws['documents']:
        d['sha256'] = digest(folder / d['stored_name'])
        if d['kind'] == 'xlsx' or '.' + d['kind'] in ocr.IMAGE_EXTENSIONS:
            continue
        max_blocks = 20000 if ws.get('pdf_import') else 2000
        bs, cs, warnings = read_document(folder / d['stored_name'], d['id'], ws['facts'], max_blocks=max_blocks)
        d['warnings'] = warnings
        blocks.extend(bs)
        for block in bs:
            cs.extend(extract_claims(block, ws['facts'], index=index))
        for c in cs:
            previous = old.get(c['id'])
            if previous:
                # 论断 ID 锚点记忆保留同位置的已确认来源; link_rules 仅为不同论断提供偏好。
                available = {f['id'] for f in ws['facts']}
                if all(r in available for r in previous['refs']):
                    c['refs'] = previous['refs']
                    c['confirmed'] = previous['confirmed']
                    c['extraction'] = previous['extraction']
            claims.append(c)
    ws['claims'], ws['blocks'] = claims, blocks
    # 级联标记只对触发它的那一次变更有效, 扫完即清, 避免后续操作误报。
    ws['graph_changed'] = []
    return ws
# ---- 事实依赖图 -----
def derivations_of(self, ws):
    return Derivations(ws.get('derivations') or [])
def build_facts(self, base, ws, previous=None, overrides=None):
    """在基础事实之上按拓扑序算出派生事实, 返回合并后的完整事实表。

```

```

论断与图表都可以引用派生事实（如「利润」「成本占比」），
因此合并结果才是 engine.check() 应该看到的完整事实集合。
"""

derivations = self.derivations_of(ws)
if not derivations.records:
    ws['graph'] = {'derived': [], 'errors': {}, 'details': {}, 'affected': [],
                  'changed': [], 'radius': 0.0, 'cycles': []}
    ws['graph_values'] = {}
    return list(base)
outcome = compute(base, derivations, changed=ws.get('graph_changed') or [],
                  overrides=overrides, previous=previous)

ws['graph'] = {
    # 派生值是 Decimal，落盘前必须转 float，否则 json.dumps 会拒绝序列化。
    'derived': [{f['id']: f['id'], 'op': f['op'], 'inputs': f['inputs'], 'unit': f['unit'],
                  'metric': f['metric'], 'value': float(f['value'])} for f in outcome['facts']]
               if f.get('derived')],
    'errors': outcome['errors'], 'details': outcome['details'],
    'affected': outcome['affected'], 'changed': outcome['changed'],
    'radius': round(outcome['radius'], 4),
    'cycles': [c for c in derivations.cycles],
}
# 保存本轮值供下一次变更比对：没有它就无法判断派生值究竟有没有变。
ws['graph_values'] = {k: (float(v) if isinstance(v, Decimal) else v)
                      for k, v in outcome['values'].items()}
return [dict(f, value=float(f['value'])) if f.get('derived') and f.get('value') is not None else f
        for f in outcome['facts']]

def save_derivations(self, ws, records):
    """替换派生定义前先完整校验，非法定义不写盘。"""
    derivations = Derivations(records)
    base_ids = {f['id'] for f in ws.get('source_facts') or ws['facts']}
    problems = derivations.validate(base_ids)
    if problems:
        raise ValueError('; '.join(problems))
    ws['derivations'] = derivations.records
    return derivations

def graph_summary(self, ws):
    graph = ws.get('graph') or {}
    return {'derived': len(graph.get('derived') or []),
            'errors': len(graph.get('errors') or {}),
            'affected': len(graph.get('affected') or []),
            'radius': graph.get('radius', 0.0),
            'cycles': len(graph.get('cycles') or [])}

def last_impact(self, ws, changed_ids):
    """本次变更的级联结果：真正变化的派生事实、直接影响与间接受影响的论断。"""
    derivations = self.derivations_of(ws)
    grouped = impact(derivations, ws['claims'], changed_ids) if derivations.records else {
        'direct': [c['id'] for c in ws['claims'] if set(c['refs']) & set(changed_ids)],
        'indirect': [], 'affected_facts': [], 'total': 0}
    graph = ws.get('graph') or {}
    return {'changed_facts': changed_ids,
            'changed_derived': graph.get('changed') or [],
            'affected_facts': grouped['affected_facts'],
            'direct_claims': grouped['direct'],
            'indirect_claims': grouped['indirect'],
            'radius': graph.get('radius', 0.0)}

```

```

def set_derivations(self, wid, revision, records):
    with self.lock:
        ws = self.read(wid)
        self.verify(ws, revision)
        self.save_derivations(ws, records)
        return self._rescan_and_commit(ws, '定义派生事实',

                                     f'共{len(ws["derivations"])}项派生事实；引用完整性校验通过')

def remove_derivation(self, wid, revision, rid):
    with self.lock:
        ws = self.read(wid)
        self.verify(ws, revision)
        derivations = self.derivations_of(ws)
        derivations.remove(rid)
        ws['derivations'] = derivations.records
        return self._rescan_and_commit(ws, '删除派生事实', rid)

def _rescan_and_commit(self, ws, event, detail):
    """派生定义变化后在原目录上重扫并落盘：不产生新 generation。"""
    self.scan(ws, self.folder(ws['id']) / ws['generation'], ws['claims'])
    self.invalidate_reviews(ws)
    ws['revision'] += 1
    ws['last_repair'] = None
    ws['audit'].append({'time': now(), 'event': event, 'detail': detail})
    self.write(ws)
    return self.public(ws)

def create(self, name, paths, demo=False, allow_empty_facts=False, pdf_import=False):
    with self.lock:
        if len(paths) < 2 or len(paths) > 10 or sum(Path(p).suffix.lower() == '.xlsx' for p in paths) != 1:
            raise ValueError('请选择一份Excel数据源和至少一份Word/PPT或图片文件，最多10份文件')
        wid = uuid.uuid4().hex
        folder = self.folder(wid)
        current = folder / 'v0'
        current.mkdir(parents=True)
        ws = {'id': wid, 'name': name.strip()[:100] or '未命名项目', 'created_at': now(),
              'revision': 0, 'generation': 'v0', 'demo': demo, 'documents': [],
              'history': [], 'audit': [], 'model_calls': [], 'last_repair': None, 'link_rules': []}
        if allow_empty_facts:
            ws['allow_empty_facts'] = True
        if pdf_import:
            ws['pdf_import'] = True
        try:
            for i, path in enumerate(paths):
                path = Path(path)
                fid = uuid.uuid4().hex[:16]
                stored = f'{fid}{path.suffix.lower()}'
                shutil.copyfile(path, current / stored)
                ws['documents'].append({'id': fid, 'name': path.name, 'kind': path.suffix.lower()[1:],
                                       'stored_name': stored, 'warnings': []})

            self.scan(ws, current)
            ws['images'], ws['ocr_blocks'], ws['ocr_claims'] = [], [], []
            for document in ws['documents']:
                if document['kind'] == 'xlsx':
                    continue
                ext = '.' + document['kind']
                if ext in ocr.IMAGE_EXTENSIONS:
                    img_id = 'img' + uuid.uuid4().hex[:16]

```

```

        mime = ocr.IMAGE_EXTENSIONS[ext]

        stored = 'images/' + img_id + '.' + ocr.MIME_EXT.get(mime, 'bin')
        path = current / stored
        path.parent.mkdir(exist_ok=True)
        shutil.copyfile(current / document['stored_name'], path)
        ws['images'].append({'id': img_id, 'file_id': document['id'],
                             'location': json.dumps(['image']), 'label': document['name'],
                             'mime': mime, 'stored_name': stored, 'sha256': digest(path),
                             'ocr_text': None, 'ocr_status': 'pending', 'ocr_error': None})
    else:
        images, warnings = read_images(current / document['stored_name'], document['id'])
        document['warnings'].extend(warnings)
        for item in images:
            stored = 'images/' + item['id'] + '.' + ocr.MIME_EXT.get(item['mime'], 'bin')
            path = current / stored
            path.parent.mkdir(exist_ok=True)
            path.write_bytes(item.pop('blob'))
            item.update(stored_name=stored, sha256=digest(path), ocr_text=None,
                        ocr_status='pending', ocr_error=None)
            ws['images'].append(item)
    if not ws['blocks'] and not ws['images']:
        raise ValueError('成果文件没有可处理的正文或幻灯片文字')
    ws['audit'].append({'time': now(), 'event': '导入文件', 'detail': f'{len(paths)}份文件；关联尚待人工确认'})
})

self.write(ws)
return self.public(ws)
except Exception:
    shutil.rmtree(folder)
    raise

def create_from_pdf(self, name, pdf_path, subject=None):
    """Create an Office-backed project from a PDF without editing the PDF.
    The generated DOCX/XLSX become the project documents. The original
    PDF is retained under ``pdf_origins`` for provenance and archive export,
    but is intentionally not scanned as an editable Office document.
    """
    source = Path(pdf_path)
    if source.suffix.lower() != '.pdf':
        raise ValueError('请选择 PDF 文件')
    if not source.is_file():
        raise FileNotFoundError(source)
    with tempfile.TemporaryDirectory() as tmp:
        backend = os.getenv('ZHILIAN_PDF_DOCX_BACKEND', '').strip().lower()
        conversion_kwargs = {'subject': subject}
        if backend:
            conversion_kwargs['docx_backend'] = backend
        conversion = export_pdf_to_office(source, tmp, **conversion_kwargs)
        if not conversion.get('facts') and not conversion.get('text_layer_pages'):
            raise ValueError('PDF 未检测到可提取文字层；网页端暂不支持扫描件 OCR，请先使用 OCR 工具处理后再导入')
    )

    generated = [Path(conversion['excel']), Path(conversion['word'])]
    manifest = dict(conversion)
    manifest.update({'source_pdf': source.name,
                    'word': Path(conversion['word']).name,

```

```

        'excel': Path(conversion['excel']).name))
    result = self.create(name, generated, allow_empty_facts=not bool(conversion.get('facts')),
                        pdf_import=True)
with self.lock:
    ws = self.read(result['id'])
    ws.setdefault('pdf_revisions', [])
    current = self.folder(ws['id']) / ws['generation']
    origin_dir = current / 'pdf_origins'
    origin_dir.mkdir(parents=True, exist_ok=True)
    source_hash = digest(source)
    safe_name = re.sub(r'[\<:\|?*\\\/]', '_', source.name).strip() or 'source.pdf'
    stored_name = f'pdf_origins/{source_hash[:16]}_{safe_name}'
    shutil.copyfile(source, current / stored_name)
    ws.setdefault('pdf_origins', []).append({
        'name': source.name, 'stored_name': stored_name,
        'sha256': source_hash, 'manifest': manifest,
    })
    ws['audit'].append({'time': now(), 'event': 'PDF 转 Office 导入',
                        'detail': f'原 PDF 保留; 生成 Word/Excel; 事实 {manifest.get("facts", 0)} 条'})
    self.write(ws)
    result = self.public(ws)
    result['pdf_conversion'] = manifest
    return result
def append_documents(self, wid, revision, paths):
    """Append a batch of result documents to an existing Excel project.
    The operation is transactional: the current generation is copied to a
    new generation, new documents are parsed there, and state is written
    only after the complete batch has scanned successfully. This keeps a
    failed large upload from leaving half an imported project.
    """
    paths = [Path(p) for p in paths]
    max_batch = int(os.getenv('ZHILIAN_MAX_APPEND_BATCH', '50'))
    max_total = int(os.getenv('ZHILIAN_MAX_PROJECT_DOCUMENTS', '500'))
    if not paths:
        raise ValueError('请至少选择一份Word、PPT或图片文件')
    if len(paths) > max_batch:
        raise ValueError(f'单批最多追加{max_batch}份文档, 请分批上传')
    allowed = {'.docx', '.pptx', *ocr.IMAGE_EXTENSIONS}
    if any(p.suffix.lower() not in allowed for p in paths):
        raise ValueError('追加接口只接受Word、PPT或图片文件, 不允许再次上传Excel')
    names = [p.name for p in paths]
    if any(not p.is_file() for p in paths):
        raise ValueError('上传文件不存在或无法读取')
    if len(set(names)) != len(names) or any(not n or len(n) > 150 or any(c in n for c in '\<:\|?*') for n in names):
        raise ValueError('文件名重复、无效或过长')
with self.lock:
    ws = self.read(wid)
    self.verify(ws, revision)
    if len(ws.get('documents', [])) + len(paths) > max_total:
        raise ValueError(f'项目文档总数不能超过{max_total}份')

    existing_names = {d['name'] for d in ws['documents']}
    if existing_names.intersection(names):
        raise ValueError('项目中已有同名文件, 请重命名后再追加')
    existing_hashes = {d.get('sha256') for d in ws['documents']}

```

```

incoming_hashes = [digest(p) for p in paths]
if any(h in existing_hashes for h in incoming_hashes):
    raise ValueError('项目中已有相同内容的文件, 请勿重复导入')
batch_id = 'b' + uuid.uuid4().hex[:16]
previous = copy.deepcopy(ws)
root = self.folder(wid)
generation = 'v' + uuid.uuid4().hex[:12]
target = root / generation
shutil.copytree(root / ws['generation'], target)
try:
    new_documents = []
    for path in paths:
        fid = uuid.uuid4().hex[:16]
        stored = f'{fid}{path.suffix.lower()}'
        shutil.copyfile(path, target / stored)
        document = {'id': fid, 'name': path.name, 'kind': path.suffix.lower()[1:],
                    'stored_name': stored, 'warnings': [], 'batch_id': batch_id}
        ws['documents'].append(document)
        new_documents.append(document)
    before_blocks = len(ws.get('blocks', []))
    before_images = len(ws.get('images', []))
    self.scan(ws, target, previous.get('claims', []))
    # Only extract assets for the newly appended documents. Existing
    # image descriptors were copied with the generation.
    for document in new_documents:
        if document['kind'] == 'xlsx':
            continue
        ext = '.' + document['kind']
        if ext in ocr.IMAGE_EXTENSIONS:
            img_id = 'img' + uuid.uuid4().hex[:16]
            mime = ocr.IMAGE_EXTENSIONS[ext]
            stored = 'images/' + img_id + '.' + ocr.MIME_EXT.get(mime, 'bin')
            image_path = target / stored
            image_path.parent.mkdir(exist_ok=True)
            shutil.copyfile(target / document['stored_name'], image_path)
            ws.setdefault('images', []).append({'id': img_id, 'file_id': document['id'],
                                                'location': json.dumps(['image']), 'label': document['name'], 'mime': mime,
                                                'stored_name': stored, 'sha256': digest(image_path), 'ocr_text': None,
                                                'ocr_status': 'pending', 'ocr_error': None})
        else:
            images, warnings = read_images(target / document['stored_name'], document['id'])
            document['warnings'].extend(warnings)
            for item in images:
                stored = 'images/' + item['id'] + '.' + ocr.MIME_EXT.get(item['mime'], 'bin')
                image_path = target / stored
                image_path.parent.mkdir(exist_ok=True)

                image_path.write_bytes(item.pop('blob'))
                item.update(stored_name=stored, sha256=digest(image_path), ocr_text=None,
                           ocr_status='pending', ocr_error=None)
                ws.setdefault('images', []).append(item)
    if len(ws.get('blocks', [])) == before_blocks and len(ws.get('images', [])) == before_images:
        raise ValueError('追加文件没有可处理的正文、幻灯片文字或图片')
    ws['generation'] = generation
    ws['revision'] += 1
    ws['last_repair'] = None

```

```

        self.invalidate_reviews(ws)
        ws.setdefault('batches', []).append({'id': batch_id, 'status': 'done',
            'created_at': now(), 'documents': [d['id'] for d in new_documents],
            'count': len(new_documents), 'revision': ws['revision']})
        ws['audit'].append({'time': now(), 'event': '追加文档',
            'detail': f'批次 {batch_id}; 新增{len(new_documents)}份文档; 等待关联确认'})
        (target / 'previous-state.json').write_text(json.dumps(previous, ensure_ascii=False), encoding='utf-
8')

        self.write(ws)
        result = self.public(ws)
        result['batch'] = {'id': batch_id, 'status': 'done', 'count': len(new_documents),
            'revision': ws['revision']}

        return result
    except Exception:
        shutil.rmtree(target, ignore_errors=True)
        raise

def batch(self, wid, batch_id):
    with self.lock:
        ws = self.read(wid)
        item = next((b for b in ws.get('batches', []) if b.get('id') == batch_id), None)
        if not item:
            raise FileNotFoundError('批次不存在')
        return copy.deepcopy(item)

def public(self, ws):
    result = copy.deepcopy(ws)
    result.setdefault('model_calls', [])
    # Do not expose request payloads or credentials. The ledger contains
    # only provider metadata, usage counters, timing and outcome.
    result['model_call_summary'] = {
        'total': len(result['model_calls']),
        'remote': sum(i.get('provider') != 'local-reranker' for i in result['model_calls']),
        'local': sum(i.get('provider') == 'local-reranker' for i in result['model_calls']),
        'succeeded': sum(i.get('outcome') == 'succeeded' for i in result['model_calls']),
        'failed': sum(i.get('outcome') in {'failed', 'invalid_response'} for i in result['model_calls']),
        'prompt_tokens': sum(int(i.get('prompt_tokens') or 0) for i in result['model_calls']),
        'completion_tokens': sum(int(i.get('completion_tokens') or 0) for i in result['model_calls']),
        'estimated_cost_cny': (round(sum(float(i['estimated_cost_cny']) for i in result['model_calls']
            if i.get('estimated_cost_cny') is not None), 8)
            if any(i.get('estimated_cost_cny') is not None for i in result['model_calls'])
            else None),
    }

    result['model_mode'] = llm.mode(ws)

    result['model_mode_label'] = llm.mode_label(result['model_mode'])
    result['model_mode_source'] = 'project' if ws.get('model_mode') in llm.LLM_MODES else 'server'
    with llm.using_mode(result['model_mode']):
        result['ocr_enabled'] = ocr.config()['enabled']
    # Pure-API comparisons must not display unconfirmed rule guesses as
    # accepted evidence. Keep stored hypotheses for switching back later.
    if result['model_mode'] == 'api_only':
        for claim in result['claims']:
            if not claim['confirmed'] and claim['kind'] != 'chart':
                claim['refs'] = []
    result['checks'], result['summary'] = inspect(result)
    result['claims'].extend(copy.deepcopy(ws.get('ocr_claims', [])))
    for claim in ws.get('ocr_claims', []):

```

```

        checked = dict(check(claim, ws['facts']), ocr=True, confirmed=claim.get('confirmed', False))
        if checked['status'] == 'unverifiable':
            checked['code'] = 'needs_review'
        result['checks'].append(checked)
        result['summary']['claims'] += 1
        result['summary'][checked['status']] += 1
        result['summary']['pending'] += not checked['confirmed']
        # 图片论断 (source='ocr') 无法回写, 不进入可修复计数。
        result['summary']['repairable'] += (not checked['ocr']) and checked['confirmed'] and checked['status'] =
= 'inconsistent'
        result['diagnosis'], result['diagnosis_summary'] = diagnose(result, result['checks'])
        # 解释仅适用于生成时的版本; 数据更新、确认、修复及撤销后使用当前模板。
        explanation_audit = next((a for a in reversed(ws['audit']) if a['event'] == '诊断解释'), {})
        if explanation_audit.get('revision') != ws['revision']:
            result['diagnosis_explanations'] = {}
        segments = unmatched_spans(ws['blocks'], ws['claims'])
        segments.extend(unmatched_spans(ws.get('ocr_blocks', []), ws.get('ocr_claims', [])))
        result['unmatched_segments'] = segments
        result['summary']['unmatched_segments'] = len(segments)
        # Keep the original block count for clients that used the first API.
        result['summary']['unmatched_blocks'] = sum(not any(c['file_id'] == b['file_id'] and c['location'] == b['loc
ation'] for c in ws['claims']) for b in ws['blocks'])
        # 派生事实与实际使用的值表是过程数据, 不重复下发给前端。
        result.pop('source_facts', None)
        result.pop('graph_values', None)
        result['summary'].update(self.graph_summary(ws))
        quality = result.get('fact_quality') or {}
        result['summary']['fact_quality_status'] = quality.get('status', 'unknown')
        result['summary']['fact_quality_issues'] = len(quality.get('issues') or [])
        for d in result['documents']:
            d.pop('stored_name', None)
            d['download_url'] = f'/api/projects/{ws["id"]}/files/{d["id"]}'
        for index, origin in enumerate(result.get('pdf_origins', [])):
            origin.pop('stored_name', None)
            origin['download_url'] = f'/api/projects/{ws["id"]}/pdf-origins/{index}'
        for index, item in enumerate(result.get('pdf_revisions', [])):
            item.pop('stored_name', None)
            item.pop('manifest_stored_name', None)
            item['download_url'] = f'/api/projects/{ws["id"]}/pdf-revisions/{index}'

            item['manifest_url'] = f'/api/projects/{ws["id"]}/pdf-revisions/{index}/manifest'
        result.setdefault('images', [])
        for item in result['images']:
            item.pop('stored_name', None)
            item['image_url'] = f'/api/projects/{ws["id"]}/images/{item["id"]}'
        return result

def set_model_mode(self, wid, revision, selected):
    if selected not in llm.LLM_MODES:
        raise ValueError('无效的模型模式')
    with self.lock:
        ws = self.read(wid)
        self.verify(ws, revision)
        previous = llm.mode(ws)
        if previous == selected and ws.get('model_mode') == selected:
            return self.public(ws)
        ws['model_mode'] = selected

```

```

ws['suggestions'] = []
ws['diagnosis_explanations'] = {}
if ws.get('agent'):
    # Retain trace and audit, but never approve stale candidates.
    ws['agent'].update(phase='idle', pending=None)
ws['revision'] += 1
self.invalidate_reviews(ws, '模型模式已变化, 请重新运行审计')
ws['audit'].append({'time': now(), 'event': '切换模型模式',
                    'detail': f'{previous} → {selected}; 旧候选待办失效, 已确认来源和文件版本保留'})
self.write(ws)
return self.public(ws)
def image_path(self, ws, image):
    """图片读取和确认前校验抽取时的摘要。"""
    path = self.folder(ws['id']) / ws['generation'] / image['stored_name']
    if digest(path) != image['sha256']:
        raise ValueError('图片在抽取后发生变化, 已停止操作, 请重新导入')
    return path
def ocr_run(self, wid, revision, image_ids=None):
    with self.lock:
        ws = self.read(wid)
        self.verify(ws, revision)
        selected = llm.mode(ws)
    with llm.using_mode(selected):
        return self._ocr_run(wid, revision, image_ids)
def _ocr_run(self, wid, revision, image_ids=None):
    with self.lock:
        ws = self.read(wid)
        self.verify(ws, revision)
        images = {i['id']: i for i in ws.get('images', [])}
        if image_ids is not None and any(i not in images for i in image_ids):
            raise ValueError('图片不存在或不属于当前项目')
        selected = [i for i in images.values() if (image_ids is None or i['id'] in image_ids)
                    and i['ocr_status'] in ('pending', 'failed')]
        settings = ocr.config()

        if not selected or not settings['enabled']:
            return self.public(ws)
        paths = {i['id']: self.image_path(ws, i) for i in selected}
# 网络请求不持锁; 提交前重新校验版本, 避免覆盖其他窗口的修改。
results = {}
for item in selected:
    gate = quota.check()
    if not gate['allowed']:
        results[item['id']] = ('failed', None, gate['reason'])
        continue
    quota.consume()
    try:
        text = ocr.ocr_image(paths[item['id']], item['mime'])
        results[item['id']] = ('done', text, None)
    except ValueError as exc:
        error = str(exc) if str(exc) in (ocr.DISABLED, ocr.UNSUPPORTED, ocr.FAILED) else ocr.FAILED
        results[item['id']] = ('failed', None, error)
with self.lock:
    ws = self.read(wid)
    self.verify(ws, revision)
    for item in ws.get('images', []):

```

```

        if item['id'] not in results:
            continue
        self.image_path(ws, item)
        item['ocr_status'], item['ocr_text'], item['ocr_error'] = results[item['id']]
        ws['audit'].append({'time': now(), 'event': 'OCR 识别',
                           'detail': f'{item["label"]}；后端 {settings["backend"]}；模型 {settings["model"]}'
                           + ('完成，等待人工核对' if item['ocr_status'] == 'done' else '失败，未
确认文字不参与验证')})
        ws['revision'] += 1
        self.write(ws)
        return self.public(ws)
def ocr_confirm(self, wid, revision, items):
    with self.lock:
        ws = self.read(wid)
        self.verify(ws, revision)
        images = {i['id']: i for i in ws.get('images', [])}
        ids = [i.get('image_id') for i in items]
        if not ids or len(set(ids)) != len(ids) or any(i not in images for i in ids):
            raise ValueError('请选择当前项目的图片，且不可重复确认')
        for item in items:
            image = images[item['image_id']]
            if image['ocr_status'] == 'pending':
                raise ValueError('请先运行 OCR，再核对文字')
            if not isinstance(item.get('text'), str) or not item['text'].strip() or len(item['text']) > ocr.MAX_
TEXT:
                raise ValueError('确认文字不能为空，且不能超过20000字')
            self.image_path(ws, image)
            blocks = [b for b in ws.get('ocr_blocks', []) if b['image_id'] not in ids]
            claims = [c for c in ws.get('ocr_claims', []) if c['image_id'] not in ids]
            for item in items:
                image = images[item['image_id']]

                text = item['text'].strip()
                block = {k: image[k] for k in ('file_id', 'location', 'label')}
                block.update(image_id=image['id'], text=text, source='ocr')
                blocks.append(block)
                for claim in extract_claims(block, ws['facts']):
                    # 图片论断只做只读核验：文字写在图片像素里，程序无法回写，
                    # 因此 repairable=False 且不计入可修复统计；人工核对的是图片文字本身，
                    # 它到事实 ID 的来源关联仍是候选，不能通过 links 接口确认，故 confirmed=False。
                    claim.update(extraction='OCR识别', source='ocr', repairable=False, image_id=image['id'], confirm
ed=False)

                    claims.append(claim)

def test_old_project_compatible(project):
    store, ws = project
    state = store.read(ws['id'])
    for key in ('images', 'ocr_blocks', 'ocr_claims'):
        state.pop(key)
    store.write(state)
    assert store.public(state)['images'] == []
    assert store.ocr_run(state['id'], state['revision'])['claims'] == state['claims']
// ===== 文件: tests/test_pdf_ingest.py =====
from zhilian.pdf_ingest import (_calculated_change, _header_columns,
                                _quarantine_conflicting_facts,

```

```

        _repair_split_numeric_cells, to_number)
def test_pdf_number_parser_handles_thousands_and_percent():
    assert to_number("1,032,869,576.20") == 1032869576.20
    assert to_number("-97.29%") == -97.29
    assert to_number("-") is None
def test_pdf_header_prefers_change_percentage_over_change_amount():
    rows = [
        ["", "2022年", "2021年", "变动金额", "变动比例(%)"],
        ["营业收入", "100", "80", "20", "25.00%"],
    ]
    current, prior, change = _header_columns(rows[:1])
    assert (current, prior, change) == (1, 2, 4)
def test_pdf_header_accepts_year_without_suffix_before_change_column():
    rows = [
        ["", "2022年", "2021", "本年比上年增减", "2020年"],
        ["营业收入", "100", "80", "25.00%", "70"]
    ]
    assert _header_columns(rows[:1]) == (1, 2, 3)
def test_pdf_repair_keeps_table_column_positions():
    row = ["资产总计", "1,", "032,869,576.20", "854,435,427.64", "20.88%"]
    fixed = _repair_split_numeric_cells(row)
    assert fixed[1] == ""
    assert fixed[2] == "1,032,869,576.20"
    assert fixed[3] == "854,435,427.64"
def test_pdf_ratio_metric_uses_percentage_point_change():
    assert _calculated_change('研发投入占营业收入比例', 3.84, 3.15, '3.84%', '3.15%') == 0.69
    assert _calculated_change('营业收入', 125, 100, '125', '100') == 25
def test_conflicting_same_page_facts_are_quarantined():
    facts = [
        {"id": "a", "source_page": 70, "metric": "营业收入", "period": "本期",
         "value": 16_456_919.36, "unit": "元", "source_table": 1, "source_row": 3},
        {"id": "b", "source_page": 70, "metric": "营业收入", "period": "本期",
         "value": 13_416_456_919.36, "unit": "元", "source_table": 2, "source_row": 2},
        {"id": "c", "source_page": 70, "metric": "营业成本", "period": "本期",

         "value": 12_610_339_015.64, "unit": "元", "source_table": 2, "source_row": 8},
    ]
    accepted, conflicts = _quarantine_conflicting_facts(facts)
    assert [item["id"] for item in accepted] == ["c"]
    assert len(conflicts) == 1
    assert conflicts[0]["metric"] == "营业收入"
    assert {item["value"] for item in conflicts[0]["candidates"]} == {
        16_456_919.36, 13_416_456_919.36,
    }
// ===== 文件: tests/test_pdf_office_export.py =====
import json
from pathlib import Path
import pytest
fitz = pytest.importorskip("fitz")
from zhilian import pdf_office_export
def _make_pdf(path):
    document = fitz.open()
    page = document.new_page()
    page.insert_text((72, 72), "Revenue for current period is 125 CNY.", fontsize=12)
    document.save(path)
    document.close()
def test_export_pdf_to_office_creates_provenance_files(tmp_path, monkeypatch):
    source = tmp_path / "annual_report.pdf"

```

```

_make_pdf(source)
monkeypatch.setattr(pdf_office_export, "_pdf_text_pages", lambda path: [
    {"page": 1, "text": "Revenue for current period is 125 CNY."}
])
monkeypatch.setattr(pdf_office_export, "extract_pdf", lambda *args, **kwargs: {
    "pages": 1, "text_layer_pages": 1, "text_layer_ratio": 1.0,
    "facts": [{"id": "pdf-pl-t1-r2-current", "metric": "营业收入", "period": "本期",
        "value": 125.0, "unit": "元", "scope": "annual_report.pdf#p1",
        "source_file": "annual_report.pdf", "source_page": 1,
        "source_table": 1, "source_row": 2}],
    "tables": [{"page": 1, "table": 1, "rows": [[["指标", "本期", "上期"]],
        "validation": {"checks": []}}],
    "chart_candidates": [],
})
manifest = pdf_office_export.export_pdf_to_office(source, tmp_path / "out")
word = tmp_path / "out" / "annual_report_正文提取.docx"
excel = tmp_path / "out" / "annual_report_事实表候选.xlsx"
assert word.exists() and excel.exists()
assert manifest["original_pdf_modified"] is False
assert manifest["requires_confirmation"] is True
assert manifest["text_anchor_count"] == 1
assert manifest["claim_candidate_count"] == 1
assert (tmp_path / "out" / "annual_report_论断候选.jsonl").read_text(encoding="utf-8").strip()
anchors = json.loads((tmp_path / "out" / "annual_report_文字锚点.json").read_text(encoding="utf-8"))
assert anchors["anchors"][0]["page"] == 1
assert "Revenue for current period" in anchors["anchors"][0]["text"]
assert json.loads((tmp_path / "out" / "annual_report_转换清单.json").read_text(encoding="utf-8"))["facts"] == 1

from openpyxl import load_workbook
workbook = load_workbook(excel, read_only=True, data_only=True)
assert workbook["事实候选"]["A2"].value == "pdf-pl-t1-r2-current"
assert workbook["事实候选"]["H2"].value == "annual_report.pdf"
assert workbook["事实候选"]["I2"].value == 1
assert workbook["事实候选"]["L2"].value == "待复核: 缺少独立交叉校验"
assert workbook["冲突待复核"].max_row == 1
workbook.close()
def test_pdf_raw_table_sheet_does_not_trigger_fact_table_row_limit(tmp_path):
    from openpyxl import Workbook
    from zhilian.office import HEADERS, read_facts
    path = tmp_path / "pdf_candidates.xlsx"
    workbook = Workbook()
    facts = workbook.active
    facts.title = "事实候选"
    facts.append(HEADERS)
    facts.append(["f1", "皖能电力", "营业收入", "本期", 100, "元", "待确认"])
    raw = workbook.create_sheet("表格原文")
    raw.append(["来源文件", "来源页", "表格编号", "行号", "列号", "原始内容"])
    for row in range(2003):
        raw.append(["report.pdf", 1, 1, row + 1, 1, "raw cell"])
    workbook.save(path)
    workbook.close()
    result = read_facts(path, "source")
    assert len(result) == 1
    assert result[0]["metric"] == "营业收入"
def test_pdf_project_endpoint_routes_to_office_conversion(tmp_path, monkeypatch):

```

```

from fastapi.testclient import TestClient
from zhilian.app import create_app
source = tmp_path / "input.pdf"
_make_pdf(source)
app = create_app(tmp_path / "api")
called = {}
def fake_create(name, path, subject=None):
    called.update(name=name, suffix=path.suffix, subject=subject, exists=path.exists())
    return {"id": "a" * 32, "name": name}
monkeypatch.setattr(app.state.store, "create_from_pdf", fake_create)
response = TestClient(app).post(
    "/api/projects/from-pdf",
    data={"name": "PDF 项目", "subject": "某公司"},
    files={"file": (source.name, source.read_bytes(), "application/pdf")},
)
assert response.status_code == 200, response.text
assert called == {"name": "PDF 项目", "suffix": ".pdf", "subject": "某公司", "exists": True}
def test_store_keeps_original_pdf_and_archives_it(tmp_path, monkeypatch):
    from zipfile import ZipFile
    from zhilian.demo import create_demo
    from zhilian.store import Store
    import zhilian.store as store_module
    source = tmp_path / "source.pdf"

    source.write_bytes(b"pdf-origin")
    def fake_export(pdf_path, output_dir, subject=None):
        paths = create_demo(output_dir)
        excel = next(path for path in paths if path.suffix == ".xlsx")
        word = next(path for path in paths if path.suffix == ".docx")
        return {"source_pdf": str(pdf_path), "word": str(word), "excel": str(excel),
                "facts": 6, "tables": 1, "chart_candidates": 0, "text_pages": 1,
                "requires_confirmation": True, "original_pdf_modified": False}
    monkeypatch.setattr(store_module, "export_pdf_to_office", fake_export)
    store = Store(tmp_path / "data")
    result = store.create_from_pdf("PDF 项目", source, subject="主体")
    state = store.read(result["id"])
    assert len(state["pdf_origins"]) == 1
    archive = store.archive(result["id"])
    with ZipFile(archive) as zipped:
        assert "原始PDF/source.pdf" in zipped.namelist()
def test_store_imports_searchable_pdf_when_no_fact_rows_are_admitted(tmp_path, monkeypatch):
    from docx import Document
    from openpyxl import Workbook
    from zhilian.office import HEADERS
    from zhilian.store import Store
    import zhilian.store as store_module
    source = tmp_path / "text_only.pdf"
    source.write_bytes(b"pdf-origin")
    def fake_export(pdf_path, output_dir, subject=None):
        output_dir = Path(output_dir)
        excel = output_dir / "candidate.xlsx"
        workbook = Workbook()
        sheet = workbook.active
        sheet.title = "事实候选"
        sheet.append(HEADERS)

```

```

        workbook.save(excel)
        word = output_dir / "body.docx"
        document = Document()
        document.add_paragraph("可提取的 PDF 正文段落。")
        document.save(word)
        return {"source_pdf": str(pdf_path), "word": str(word), "excel": str(excel),
                "facts": 0, "tables": 1, "chart_candidates": 0, "text_pages": 1,
                "text_layer_pages": 1, "requires_confirmation": True,
                "original_pdf_modified": False}

monkeypatch.setattr(store_module, "export_pdf_to_office", fake_export)
store = Store(tmp_path / "data")
result = store.create_from_pdf("PDF 项目", source)
state = store.read(result["id"])
assert state["facts"] == []
assert state["allow_empty_facts"] is True
assert state["fact_quality"]["status"] == "blocked"
assert len(state["pdf_origins"]) == 1

def test_store_exports_independent_revised_pdf_and_manifest(tmp_path, monkeypatch):
    from zhilian.demo import create_demo

    from zhilian.store import Store
    import zhilian.store as store_module
    source = tmp_path / "source.pdf"
    source.write_bytes(b"pdf-origin")
    def fake_export(pdf_path, output_dir, subject=None):
        paths = create_demo(output_dir)
        excel = next(path for path in paths if path.suffix == ".xlsx")
        word = next(path for path in paths if path.suffix == ".docx")
        return {"source_pdf": str(pdf_path), "word": str(word), "excel": str(excel),
                "facts": 6, "tables": 1, "chart_candidates": 0,
                "requires_confirmation": True, "original_pdf_modified": False}
    def fake_docx_to_pdf(docx, output):
        output.write_bytes(b"revised-pdf")
        return {"docx": str(docx), "pdf": str(output), "backend": "test",
                "docx_sha256": "docx", "pdf_sha256": "pdf", "original_is_read_only": True}
    def fake_verify(original, revised):
        return {"passed": True, "status": "passed", "page_count": {"unchanged": True}}
    def fake_manifest(original, exported, verification, output):
        output.write_text(json.dumps(verification), encoding="utf-8")
        return verification
    monkeypatch.setattr(store_module, "export_pdf_to_office", fake_export)
    monkeypatch.setattr(store_module, "export_docx_to_pdf", fake_docx_to_pdf)
    monkeypatch.setattr(store_module, "verify_revised_pdf", fake_verify)
    monkeypatch.setattr(store_module, "compare_pdf_visual_fidelity", lambda original, revised, **kwargs: {
        "passed": True, "status": "passed", "changed_page_count": 0,
        "page_count": {"original": 1, "revised": 1, "unchanged": True},
    })
    monkeypatch.setattr(store_module, "build_revised_pdf_manifest", fake_manifest)
    store = Store(tmp_path / "data")
    result = store.create_from_pdf("PDF 项目", source, subject="主体")
    state = store.read(result["id"])
    revised = store.export_revised_pdf(result["id"], state["revision"])
    saved = store.read(result["id"])
    assert revised["pdf_revision"]["verification"]["status"] == "passed"
    assert revised["pdf_revision"]["verification"]["visual_check"]["passed"] is True

```

```
    assert len(saved["pdf_revisions"]) == 1
    assert (store.folder(result["id"]) / saved["generation"] / saved["pdf_revisions"][0]["stored_name"]).read_bytes(
) == b"revised-pdf"
// ===== 文件: tests/test_pdf_overlay_experiment.py =====
import pytest
fitz = pytest.importorskip("fitz")
from zhilian.pdf_overlay_experiment import overlay_replace_pdf
from zhilian.pdf_writeback import verify_pdf_writeback
def _make_pdf(path):
    document = fitz.open()
    page = document.new_page()
    page.insert_text((72, 72), "Revenue for current period is 125 CNY.", fontsize=12)
    document.save(path)
    document.close()
def test_visual_overlay_writes_a_new_pdf_and_replaces_text(tmp_path):
    source = tmp_path / "source.pdf"

    output = tmp_path / "overlay.pdf"
    _make_pdf(source)
    result = overlay_replace_pdf(source, output, page_number=1, old="125", new="130")
    assert result["mode"] == "visual_overlay"
    assert output.exists()
    with fitz.open(output) as document:
        text = document[0].get_text("text")
    assert "130" in text
    assert "125" not in text
    verified = verify_pdf_writeback(source, output, [{"page": 1, "old": "125", "new": "130"}])
    assert verified["passed"] is True
def test_visual_overlay_rejects_missing_text(tmp_path):
    source = tmp_path / "source.pdf"
    output = tmp_path / "overlay.pdf"
    _make_pdf(source)
    with pytest.raises(ValueError, match="找不到"):
        overlay_replace_pdf(source, output, page_number=1, old="999", new="130")
// ===== 文件: tests/test_pdf_revised_export.py =====
import json
import pytest
fitz = pytest.importorskip("fitz")
from zhilian.pdf_revised_export import (available_docx_backends, build_revised_pdf_manifest,
                                         choose_pdf_docx_backend, compare_pdf_visual_fidelity,
                                         verify_revised_pdf)
def _make_pdf(path, pages):
    document = fitz.open()
    for text in pages:
        page = document.new_page()
        page.insert_text((72, 72), text, fontsize=12)
    document.save(path)
    document.close()
def test_verify_revised_pdf_reopens_output_and_preserves_original(tmp_path):
    original = tmp_path / "original.pdf"
    revised = tmp_path / "revised.pdf"
    _make_pdf(original, ["Original text 125", "Keep unchanged"])
    _make_pdf(revised, ["Revised text 130", "Keep unchanged"])
    before = original.read_bytes()
    result = verify_revised_pdf(original, revised, expected_texts=["Revised text 130"])
```

```

    assert result["passed"] is True
    assert result["page_count"]["unchanged"] is True
    assert result["expected_texts"]["Revised text 130"] is True
    assert original.read_bytes() == before
def test_manifest_records_independent_revision_and_hashes(tmp_path):
    original = tmp_path / "original.pdf"
    revised = tmp_path / "revised.pdf"
    _make_pdf(original, ["Original text"])
    _make_pdf(revised, ["Revised text"])
    verification = verify_revised_pdf(original, revised)
    manifest_path = tmp_path / "revision.json"
    manifest = build_revised_pdf_manifest(

        original,
        {"pdf": str(revised), "docx": "edited.docx", "backend": "microsoft_word"},
        verification,
        manifest_path,
    )
    assert manifest["original_is_read_only"] is True
    assert manifest["original_pdf_sha256"]
    assert json.loads(manifest_path.read_text(encoding="utf-8"))["verification"]["passed"] is True
def test_backend_capability_detection_is_explicit(monkeypatch):
    backends = available_docx_backends()
    assert set(("native", "pdf2docx", "microsoft_word", "libreoffice")) <= set(backends)
    assert backends["native"]["available"] is True
    monkeypatch.setattr("zhilian.pdf_revised_export.available_docx_backends",
                        lambda: {"native": {"available": True}, "pdf2docx": {"available": False}})
    assert choose_pdf_docx_backend("auto") == "native"
def test_visual_fidelity_gate_passes_for_identical_render_and_writes_report(tmp_path):
    original = tmp_path / "original.pdf"
    revised = tmp_path / "revised.pdf"
    _make_pdf(original, ["Same text"])
    original_bytes = original.read_bytes()
    revised.write_bytes(original_bytes)
    report = compare_pdf_visual_fidelity(original, revised, dpi=72)
    assert report["passed"] is True
    assert report["changed_page_count"] == 0
    assert report["pages"][0]["text_exact"] is True
def test_visual_fidelity_gate_reports_changed_page_and_bbox(tmp_path):
    original = tmp_path / "original.pdf"
    revised = tmp_path / "revised.pdf"
    _make_pdf(original, ["Original text"])
    _make_pdf(revised, ["Changed text"])
    report = compare_pdf_visual_fidelity(original, revised, dpi=72)
    assert report["passed"] is False
    assert report["status"] == "needs_visual_review"
    assert report["changed_page_count"] == 1
    assert report["pages"][0]["bbox"] is not None
    assert report["pages"][0]["text_exact"] is False
// ===== 文件: tests/test_pdf_writeback.py =====
import pytest
fitz = pytest.importorskip("fitz")
from zhilian import pdf_writeback
def _make_pdf(path, pages):
    document = fitz.open()

```

```

for text in pages:
    page = document.new_page()
    page.insert_text((72, 72), text, fontsize=12)
document.save(path)
document.close()
def test_preflight_defaults_to_errata_when_font_coverage_is_unknown(tmp_path):
    source = tmp_path / "source.pdf"
    _make_pdf(source, ["Revenue for current period is 125 CNY."])

    result = pdf_writeback.preflight_pdf_writeback(
        source,
        [{"page": 1, "old": "125", "new": "130", "anchor": "营收"}],
    )
    assert result["decision"] == "errata_only"
    assert result["text_layer"]["has_text_layer"] is True
    assert result["occurrences"][0]["status"] == "unique_target"
    assert result["font"]["status"] == "unknown"
    assert result["policy"]["original_is_never_overwritten"] is True
def test_duplicate_old_value_rejects_body_only_writeback(tmp_path):
    source = tmp_path / "duplicate.pdf"
    _make_pdf(source, ["Revenue for current period is 125 CNY.", "Table Revenue 125 100 25 25%"])
    result = pdf_writeback.preflight_pdf_writeback(
        source,
        [{"page": 1, "old": "125", "new": "130"}],
    )
    assert result["decision"] == "reject"
    assert any("位置不唯一" in reason for reason in result["reasons"])
def test_equivalent_table_value_also_rejects_body_only_writeback(tmp_path):
    source = tmp_path / "equivalent.pdf"
    _make_pdf(source, ["Revenue 6.04 CNY hundred million.",
        "Table Revenue 604,539,544.17 CNY."])
    result = pdf_writeback.preflight_pdf_writeback(
        source,
        [{"page": 1, "old": "6.04", "new": "5.80",
            "equivalent_values": ["604,539,544.17"]}],
    )
    assert result["decision"] == "reject"
    assert result["occurrences"][0]["equivalent_outside_target_pages"] == [2]
def test_source_fact_automatically_finds_scaled_and_rounded_table_value(tmp_path):
    source = tmp_path / "auto-equivalent.pdf"
    _make_pdf(source, ["Revenue is 6.04 billion CNY.",
        "Table Revenue 604,539,544.17 CNY."])
    result = pdf_writeback.preflight_pdf_writeback(
        source,
        [{"page": 1, "old": "6.04", "new": "5.80",
            "source_value": 604539544.17, "source_unit": "元"}],
    )
    assert result["decision"] == "reject"
    assert result["source_rendering"]["generated_variant_count"] > 0
    assert result["occurrences"][0]["equivalent_outside_target_pages"] == [2]
def test_scanned_or_empty_pdf_rejects_writeback(tmp_path):
    source = tmp_path / "scan.pdf"
    document = fitz.open()
    document.new_page()
    document.save(source)

```

```

document.close()
result = pdf_writeback.preflight_pdf_writeback(
    source,
    [{"page": 1, "old": "125", "new": "130"}],

)
assert result["decision"] == "reject"
assert any("文字层" in reason for reason in result["reasons"])
def test_table_signal_rejects_body_only_writeback(tmp_path, monkeypatch):
    source = tmp_path / "table.pdf"
    _make_pdf(source, ["Revenue table 125 100 25% Profit 8 6 33%"])
    monkeypatch.setattr(pdf_writeback, "_page_visual_signals", lambda *args, **kwargs: {
        "table_candidate": True, "chart_candidate": False, "numeric_count": 6,
        "images": 0, "drawings": 0,
    })
    result = pdf_writeback.preflight_pdf_writeback(
        source,
        [{"page": 1, "old": "125", "new": "130"}],
    )
    assert result["decision"] == "reject"
    assert any("表格/图表" in reason for reason in result["reasons"])
def test_writeback_requires_explicit_user_confirmation_and_switch(tmp_path, monkeypatch):
    source = tmp_path / "confirmed.pdf"
    _make_pdf(source, ["Revenue for current period is 125 CNY."])
    monkeypatch.setattr(pdf_writeback, "_font_coverage", lambda *args, **kwargs: {
        "status": "safe", "required_glyphs": ["1", "3", "0"], "missing_glyphs": []
    })
    monkeypatch.setattr(pdf_writeback, "_signature_evidence", lambda *args, **kwargs: {
        "status": "safe", "detected": False, "evidence": []
    })
    pending = pdf_writeback.preflight_pdf_writeback(
        source, [{"page": 1, "old": "125", "new": "130", "source_value": 125, "source_unit": "元"}], user_confirmed=
False,
        allow_writeback=True,
    )
    assert pending["decision"] == "errata_only"
    allowed = pdf_writeback.preflight_pdf_writeback(
        source, [{"page": 1, "old": "125", "new": "130", "source_value": 125, "source_unit": "元"}], user_confirmed=
True,
        allow_writeback=True,
    )
    assert allowed["decision"] == "writeback_allowed"
def test_verify_requires_page_count_and_untouched_page_hashes(tmp_path):
    original = tmp_path / "original.pdf"
    modified = tmp_path / "modified.pdf"
    _make_pdf(original, ["Revenue for current period is 125 CNY.", "Second page stays unchanged."])
    _make_pdf(modified, ["Revenue for current period is 130 CNY.", "Second page stays unchanged."])
    result = pdf_writeback.verify_pdf_writeback(
        original, modified, [{"page": 1, "old": "125", "new": "130"}]
    )
    assert result["passed"] is True
    assert result["page_count"]["unchanged"] is True
    assert result["untouched_hashes_unchanged"] is True
    assert result["changes"][0]["new_independently_extractable"] is True
def test_errata_markdown_keeps_original_untouched(tmp_path):

```

```

source = tmp_path / "source.pdf"
_make_pdf(source, ["Revenue for current period is 125 CNY."])

preflight = pdf_writeback.preflight_pdf_writeback(
    source, [{"page": 1, "old": "125", "new": "130"}]
)
markdown = pdf_writeback.build_errata_markdown(
    preflight, [{"page": 1, "old": "125", "new": "130", "source": "事实表 A1"}]
)
assert "原 PDF 未被修改" in markdown
assert "125" in markdown and "130" in markdown
assert source.exists()
// ===== 文件: tests/test_promote_approved_pairs.py =====
import json
from pathlib import Path
import pytest
from 答辩评测.promote_approved_pairs import main
def write_dataset(root: Path, *, ready=True, labels=True, balanced=True):
    root.mkdir()
    (root / "manifest.json").write_text(
        json.dumps({"ready_for_training": ready}, ensure_ascii=False), encoding="utf-8"
    )
    for split in ("train", "dev", "test"):
        rows = [
            {
                "claim_id": f"{split}-claim",
                "claim_text": "营收为一百万元",
                "fact_text": f"metric=营业收入; period={'本期' if split != 'test' else '上期'}",
                "group_id": split,
                "label": 1 if labels else None,
                "period": "本期" if split != "test" else "上期",
            },
            {
                "claim_id": f"{split}-claim",
                "claim_text": "营收为一百万元",
                "fact_text": "metric=营业成本; period=本期",
                "group_id": split,
                "label": 0 if labels and balanced else None,
                "period": "本期",
            },
        ]
    if split == "test":
        rows.append({
            "claim_id": f"{split}-claim",
            "claim_text": "营收为一百万元",
            "fact_text": "metric=营业收入; period=本期",
            "group_id": split,
            "label": 1 if labels else None,
            "period": "本期",
        })
    (root / f"{split}.jsonl").write_text(
        "".join(json.dumps(row, ensure_ascii=False) + "\n" for row in rows),
        encoding="utf-8",

```

```

    )
def run(source: Path, out: Path) -> int:
    import sys
    old = sys.argv
    try:
        sys.argv = ["promote", "--source", str(source), "--out", str(out)]
        return main()
    finally:
        sys.argv = old
def test_rejects_unapproved_without_creating_output(tmp_path):
    source = tmp_path / "source"
    write_dataset(source, ready=False)
    out = tmp_path / "out"
    assert run(source, out) == 2
    assert not out.exists()
def test_rejects_pending_labels_without_creating_output(tmp_path):
    source = tmp_path / "source"
    write_dataset(source, ready=True, labels=False)
    out = tmp_path / "out"
    assert run(source, out) == 2
    assert not out.exists()
def test_promotes_approved_balanced_data(tmp_path):
    source = tmp_path / "source"
    write_dataset(source, ready=True)
    out = tmp_path / "out"
    assert run(source, out) == 0
    manifest = json.loads((out / "manifest.json").read_text(encoding="utf-8"))
    assert manifest["ready_for_training"] is True
    assert all((out / f"{split}.jsonl").is_file() for split in ("train", "dev", "test"))
// ===== 文件: tests/test_quota.py =====
"""模型调用限额: 按访客与全局双闸门, 超限降级为规则模式而不是报错。"""
import json
from datetime import date, timedelta
import httpx
import pytest
from fastapi.testclient import TestClient
from zhilian import agent, quota
from zhilian.app import create_app
from zhilian.demo import create_demo
from zhilian.store import Store
@pytest.fixture(autouse=True)
def isolated(tmp_path, monkeypatch):
    monkeypatch.setenv('DEEPSEEK_API_KEY', '')
    monkeypatch.setenv('DEEPSEEK_MODEL', 'deepseek-flash')
    monkeypatch.setattr('zhilian.app.load_environment', lambda: None)
    quota.configure(tmp_path / 'quota.json')
    token = quota.identify('测试访客')
    yield
    quota.release(token)
@pytest.fixture
def project(tmp_path):
    store = Store(tmp_path / 'data')
    return store, store.create('限额测试', create_demo(tmp_path / 'files'), True)
def force_hard_claims(monkeypatch):

```

```

"""把全部论断当作困难样本，确保走到远程模型这一段。"""
routing = agent._candidate_buckets
def all_hard(ws, rules):
    unique, multi, zero = routing(ws, rules)
    return [], [], unique + multi + zero
monkeypatch.setattr(agent, '_candidate_buckets', all_hard)
def fake_api(monkeypatch, calls):
    def post(url, **kwargs):
        calls.append(url)
        payload = {'suggestions': [{'claim_id': kwargs['json']['messages'][1]['content'][:0] or 'x',
                                   'refs': [], 'reason': ''}]}
        return httpx.Response(200, json={'choices': [{'message': {'content': json.dumps(payload)}}]})
    monkeypatch.setattr(httpx, 'post', post)
def test_per_client_limit_degrades_to_rules(project, tmp_path, monkeypatch):
    store, ws = project
    monkeypatch.setenv('DEEPSEEK_API_KEY', 'test-secret')
    monkeypatch.setenv('ZHILIAN_QUOTA_PER_CLIENT', '1')
    monkeypatch.setenv('ZHILIAN_QUOTA_GLOBAL', '50')
    force_hard_claims(monkeypatch)
    calls = []
    fake_api(monkeypatch, calls)
    ws = agent.run_agent(store, ws['id'], ws['revision'])
    assert len(calls) == 1, '首次应调用模型'
    assert ws['agent']['model_routing']['model_quota']['client_used'] == 1
    assert ws['agent']['model_routing']['api_batches'] == 1
    # 同一访客再开一个新项目：额度已用尽，必须降级但仍走完流程。
    # （对同一项目重复 run 本来就会复用已有候选，不会重新路由。）
    second = store.create('限额测试·第二个项目', create_demo(tmp_path / 'files2'), True)
    before = len(calls)
    ws = agent.run_agent(store, second['id'], second['revision'])
    assert len(calls) == before, '额度用尽后不应再发起外部请求'
    assert any(a['event'] == '模型额度用尽' for a in ws['audit'])
    assert any(t['node'] == 'model_skipped' and '额度' in t['summary'] for t in ws['agent']['trace'])
    assert ws['agent']['model_routing']['model_quota']['client_remaining'] == 0
    assert ws['agent']['model_routing']['api_batches'] == 0
    assert ws['agent']['pending'], '仍应把候选交给人工确认，而不是卡住'
def test_global_limit_blocks_every_visitor(project, monkeypatch):
    store, ws = project
    monkeypatch.setenv('DEEPSEEK_API_KEY', 'test-secret')
    monkeypatch.setenv('ZHILIAN_QUOTA_PER_CLIENT', '100')
    monkeypatch.setenv('ZHILIAN_QUOTA_GLOBAL', '1')
    force_hard_claims(monkeypatch)
    calls = []
    fake_api(monkeypatch, calls)
    ws = agent.run_agent(store, ws['id'], ws['revision'])
    assert len(calls) == 1

# 换一个访客也应被全局预算拦住
token = quota.identify('另一个访客')
try:
    ws = agent.run_agent(store, ws['id'], ws['revision'])
    assert len(calls) == 1, '全局预算用尽后任何访客都不应再发起请求'
    assert quota.check()['code'] == 'global_exhausted'
finally:
    quota.release(token)

```

```

def test_quota_counts_per_visitor_and_resets_next_day(tmp_path, monkeypatch):
    monkeypatch.setenv('ZHILIAN_QUOTA_PER_CLIENT', '2')
    quota.consume('访客甲')
    quota.consume('访客甲')
    assert quota.check('访客甲')['code'] == 'client_exhausted'
    assert quota.check('访客乙')['allowed'] is True, '额度必须按访客隔离'
    assert quota.snapshot('访客甲')['global_used'] == 2
    # 模拟跨天: 把日期改成昨天, 计数应清零
    path = tmp_path / 'quota.json'
    data = json.loads(path.read_text(encoding='utf-8'))
    data['day'] = (date.today() - timedelta(days=1)).isoformat()
    path.write_text(json.dumps(data), encoding='utf-8')
    assert quota.snapshot('访客甲')['client_used'] == 0
    assert quota.snapshot('访客甲')['global_used'] == 0

def test_quota_file_is_never_a_single_point_of_failure(tmp_path, monkeypatch):
    """文件损坏或不可写时按零消耗处理, 绝不能影响演示。"""
    monkeypatch.setenv('ZHILIAN_QUOTA_PER_CLIENT', '5')
    quota.configure(tmp_path / 'quota.json')
    (tmp_path / 'quota.json').write_text('{ 这不是 JSON', encoding='utf-8')
    assert quota.check('访客')['allowed'] is True
    quota.consume('访客')
    assert quota.snapshot('访客')['client_used'] == 1

def test_quota_endpoint_and_health_expose_limits(tmp_path, monkeypatch):
    monkeypatch.delenv('ZHILIAN_ACCESS_PASSWORD', raising=False)
    monkeypatch.setenv('ZHILIAN_QUOTA_PER_CLIENT', '30')
    monkeypatch.setenv('ZHILIAN_QUOTA_GLOBAL', '300')
    app = create_app(tmp_path / 'api')
    client = TestClient(app)
    health = client.get('/api/health').json()
    assert health['quota']['per_client_limit'] == 30 and health['quota']['global_limit'] == 300
    state = client.get('/api/quota').json()
    assert state['client_remaining'] == 30 and state['global_remaining'] == 300

def test_visitor_identity_follows_cloudflare_header():
    """演示环境在 Cloudflare 之后, 必须用 CF-Connecting-IP 而不是边缘节点地址。"""
    headers = {'cf-connecting-ip': '203.0.113.9', 'x-forwarded-for': '198.51.100.7'}
    assert quota.client_key(headers, '10.0.0.1') == '203.0.113.9'
    assert quota.client_key({'x-forwarded-for': '198.51.100.7', '10.0.0.1'}, '10.0.0.1') == '198.51.100.7'
    assert quota.client_key({}, '127.0.0.1') == '127.0.0.1'

// ===== 文件: tests/test_real_phrasing.py =====
"""真实中文语料表达方式: 年报用词、千分位数值、动词变体。
背景: 引擎在项目自建的合成语料上识别率 100%, 但在真实上市公司年报上
识别率约 0.5% (580 条含数字句子中仅 3 条)。本文件锁定已补齐的那部分能力,

```

并明确记录仍未覆盖的部分, 避免再次出现「合成语料 100% = 真实泛化能力」的误判。

"""

```

import pytest
from zhilian.engine import best_facts, check, extract_claims, number
def kinds(text, facts=()):
    return [c['kind'] for c in extract_claims(
        {'file_id': 'F', 'location': ['p', 0], 'label': 't', 'text': text}, list(facts))]
# ----- 千分位数值 -----
@pytest.mark.parametrize('raw, expected', [
    ('3,379.37', '3379.37'),
    ('1,234,567.89', '1234567.89'),
    ('-1,000', '-1000'),

```

```

    ('125', '125'),
    ('0.5', '0.5'),
])
def test_thousand_separator_is_parsed(raw, expected):
    assert str(number(raw)) == expected
@pytest.mark.parametrize('bad', ['1,23', ',123', '1,', '12,3456'])
def test_malformed_separator_is_rejected(bad):
    """逗号不按三位分组时不剥离，避免把异常输入悄悄改成一个别的数。"""
    with pytest.raises(ValueError):
        number(bad)
def test_quote_with_thousand_separator_is_extracted():
    assert kinds('报告期内投资收益为3,379.37万元') == ['quote']
    claim = extract_claims(
        {'file_id': 'F', 'location': ['p', 0], 'label': 't', 'text': '投资收益为3,379.37万元'}, [])[0]
    assert claim['spec']['reported'] == pytest.approx(3379.37)
@pytest.mark.parametrize('text,value,status', [
    ('本期经营活动产生的现金流量净额为13.67亿元', '1367348095.07', 'consistent'),
    ('本期经营活动产生的现金流量净额为13.68亿元', '1367348095.07', 'inconsistent'),
    ('本期经营活动产生的现金流量净额为13.67亿元', '1367500000', 'inconsistent'),
    ('本期经营活动产生的现金流量净额为1,367,348,095.07元', '1367348095.08', 'inconsistent'),
    ('本期经营活动产生的现金流量净额为-13.67亿元', '-1367348095.07', 'consistent'),
])
def test_original_display_precision_and_currency_conversion(text, value, status):
    facts = [{'id': 'f', 'subject': '总计', 'metric': '经营活动产生的现金流量净额',
              'period': '本期', 'unit': '元', 'value': value, 'scope': '合并'}]
    c = extract_claims({'file_id': 'F', 'location': ['p', 0], 'label': '正文', 'text': text}, facts)[0]
    assert check(c, facts)['status'] == status
@pytest.mark.parametrize('text', [
    '公司本期实现营收125万元',
    '营业收入125万元',
    '本期销售额录得125万元',
    '本期销售额共计125万元',
    '本期销售额增至125万元',
    '本期销售额高达125万元',
    '本期销售额约为125万元',
    '本期销售额累计125万元',
])
def test_quote_vocabulary_variants_are_recognized(text):
    """年报常见连接词和标题式金额应进入论断层。"""
    assert kinds(text) == ['quote']
# ---- 增长率用词 -----
@pytest.mark.parametrize('text', [
    '较上期增长25%', # 原有表达，必须保持可用
    '较上期下降10%',
    '营业收入同比增长25%', # 年报常用「同比」
    '营业收入较上年增加553.21%', # 年报用「较上年」+「增加」
    '营业成本较上年减少12.5%',
    '营业收入比上年上升8%',
    '公司产品长链二元酸产量增加179.79%',
])
def test_growth_vocabulary_variants_are_recognized(text):
    assert kinds(text) == ['growth']
@pytest.mark.parametrize('text,direction', [
    ('营业收入同比增长25%', 1),

```

```

    ('营业收入较上年增加553.21%', 1),
    ('营业收入比上年上升8%', 1),
    ('营业成本较上年减少12.5%', -1),
    ('较上期下降10%', -1),
    ('较上期持平', 0),
])
def test_growth_direction_is_mapped_from_vocabulary(text, direction):
    """方向词可能是否定含义的动词，必须映射正确，否则会把下降读成增长。"""
    spec = extract_claims(
        {'file_id': 'F', 'location': ['p', 0], 'label': 't', 'text': text}, []) [0] ['spec']
    assert spec['direction'] == direction
def test_negative_direction_reports_negative_value():
    spec = extract_claims(
        {'file_id': 'F', 'location': ['p', 0], 'label': 't',
         'text': '营业成本较上年减少12.5%'}, []) [0] ['spec']
    assert spec['reported'] < 0
def test_qualitative_growth_still_supported():
    spec = extract_claims(
        {'file_id': 'F', 'location': ['p', 0], 'label': 't',
         'text': '营业收入同比增长'}, []) [0] ['spec']
    assert spec['qualitative'] is True
@pytest.mark.parametrize('text', ['营业收入同比减少573.71万元', '营业收入同比增长率25%',
                                   '营业收入增加一倍', '提高了服务质量'])
def test_amount_changes_are_not_reported_as_qualitative_percentage_growth(text):
    assert 'growth' not in kinds(text)
def test_bare_percentage_requires_user_to_complete_comparison_period():
    facts = [{'id': p, 'subject': '总计', 'metric': '营业收入', 'period': p,
              'value': 100, 'unit': '万元', 'scope': '合并'} for p in ('本期', '上期')]
    claim = extract_claims({'file_id': 'F', 'location': ['p', 0], 'label': 't',
                           'text': '营业收入增加25%'}, facts) [0]
    assert claim['kind'] == 'growth' and claim['spec'] ['reported'] == 25
    assert claim['refs'] == [] and claim['confirmed'] is False
    assert claim['spec'] ['comparison_period_explicit'] is False
    assert '比较基准未明确' in claim['issue']

# ---- 阈值用词 -----
@pytest.mark.parametrize('text', [
    '支出未超过预算',
    '本期销售额超过120万元',
    '营业收入占比超过50%',
    '支出不少于100万元',
    '本期支出低于100万元',
])
def test_threshold_variants_are_recognized(text):
    assert kinds(text) == ['threshold']
def test_threshold_limit_with_separator():
    spec = extract_claims(
        {'file_id': 'F', 'location': ['p', 0], 'label': 't',
         'text': '本期销售额超过1,200万元'}, []) [0] ['spec']
    assert spec['limit'] == pytest.approx(1200.0)
@pytest.mark.parametrize('metric, phrase', [('营业收入', '营收'), ('资产总计', '总资产'), ('负债总计', '总负债'), ('员工人数', '员工总数')])
def test_metric_aliases_recall_canonical_fact(metric, phrase):
    facts = [{'id': 'f1', 'subject': '公司', 'metric': metric, 'period': '本期',
              'value': 125, 'unit': '万元', 'scope': '合计'}]

```

```

    assert [f['id'] for f in best_facts(f' 公司本期实现 {phrase} 125万元', facts)] == ['f1']
@pytest.mark.parametrize('metric, phrase', [( '营业收入', '主营业务收入'), ( '研发费用', '研发投入'), ( '货币资金', '现金及
现金等价物')])
def test_accounting_scopes_are_not_aliases(metric, phrase):
    facts = [{'id': 'f1', 'subject': '公司', 'metric': metric, 'period': '本期'}]
    assert best_facts(f' 公司本期 {phrase}', facts) == []
# ---- 已知缺口: 明确记录为「尚未支持」-----
@pytest.mark.parametrize('text, reason', [
    ( '营业收入占比已超过公司全部营业收入的50%以上', '算子与数值之间隔着较长定语'),
    ( '本期毛利率低于去年同期', '与历史期间比较但无数值'),
    ( '生物基、淀粉基新材增加187.44', '表格残片, 无单位'),
])
def test_known_gaps_are_documented_not_silently_wrong(text, reason):
    """这些是真实年报里的表达, 当前引擎识别不到。
    这里断言「识别不到」而非「识别对了」, 是为了把缺口固定在测试里:
    一旦后续补齐, 本测试会失败, 从而提醒同步更新文档与评测口径。
    """
    assert kinds(text) == [], f' 缺口已变化 ({reason}), 请更新文档与评测口径'
def test_uncovered_metrics_do_not_inherit_cashflow_source():
    facts = [{'id': 'cash', 'subject': '总计', 'metric': '经营活动产生的现金流量净额',
              'period': '本期', 'value': 1367348095.07, 'unit': '元', 'scope': '合并'}]
    text = ( '报告期内公司经营活动产生的现金流量净额为1,367,348,095.07元, '
            '本年度净利润为639,663,726.67元, 差异727,684,368.40元。')
    claims = extract_claims({'file_id': 'F', 'location': ['p', 0], 'label': '正文', 'text': text}, facts)
    assert [c['refs'] for c in claims] == [['cash'], [], []]
def test_new_metric_does_not_inherit_growth_source():
    facts = [{'id': p, 'subject': '总计', 'metric': '营业收入', 'period': p,
              'value': v, 'unit': '万元', 'scope': '合并'} for p, v in [( '本期', 125), ( '上期', 100)]]
    for text in [ '本期营业收入为125万元, 净利润同比增长25%。',
                  '本期营业收入为125万元。净利润同比增长25%。']:
        claims = extract_claims({'file_id': 'F', 'location': ['p', 0], 'label': '正文', 'text': text}, facts)
        assert next(c for c in claims if c['kind'] == 'growth')['refs'] == []

// ===== 文件: tests/test_report.py =====
from copy import deepcopy
from zipfile import ZipFile
import httpx
import pytest
from fastapi.testclient import TestClient
from openpyxl import load_workbook
from zhilian import app as app_module
from zhilian.demo import create_demo
from zhilian.report import build_report
from zhilian.store import Store
@pytest.fixture(autouse=True)
def isolated_environment(monkeypatch):
    monkeypatch.setattr(app_module, 'load_environment', lambda: None)
    monkeypatch.setenv('DEEPSEEK_API_KEY', 'test-secret')
    monkeypatch.setenv('ZHILIAN_ACCESS_PASSWORD', '')
    # 固定生成时刻以比较接口、导出和直接生成的全文。
    monkeypatch.setattr('zhilian.report.now', lambda: '2026-09-20T06:00:00+00:00')
    def forbidden(*args, **kwargs):
        pytest.fail('生成报告不应调用外部模型')
    monkeypatch.setattr(httpx, 'post', forbidden)
@pytest.fixture

```

```

def project(tmp_path):
    store = Store(tmp_path / 'data')
    return store, store.create('成果报告测试', create_demo(tmp_path / 'files'), True)

def confirm_all(store, ws):
    return store.confirm(ws['id'], ws['revision'], [
        {'claim_id': c['id'], 'refs': c['refs']} for c in ws['claims']]

def test_report_contains_changes_comparison_evidence_and_todos(project):
    store, ws = project
    ws = store.change(ws['id'], ws['revision'], {'sales_current': 90})
    claim = next(c for c in ws['claims'] if c['kind'] == 'quote')
    ws = store.confirm(ws['id'], ws['revision'], [{'claim_id': claim['id'], 'refs': claim['refs']}])
    ws = store.repair(ws['id'], ws['revision'], [claim['id']])
    before = deepcopy(ws)
    report = build_report(ws)
    for value in ('# 知链成果与变更报告', '销售额', 'sales_current: 125.0 → 90.0',
        '修改前后对照', '原文: 本期销售额为125万元', '改为: 本期销售额为90万元',
        '依据:', '事实表!E3', '演示业务', '人工待办', '未确认来源', '本次修复 1 项'):
        assert value in report
    assert ws == before

def test_empty_states_and_completed_project(project):
    store, ws = project
    report = build_report(ws)
    assert '本次无数据变更' in report and '本次无文件修复' in report
    assert '无待办, 可交付' not in report
    ws = confirm_all(store, ws)
    assert '无待办, 可交付' in build_report(ws)

def test_confirmed_but_unrepaired_is_not_deliverable(project):
    store, ws = project

    ws = confirm_all(store, ws)
    ws = store.change(ws['id'], ws['revision'], {'sales_current': 90})
    report = build_report(ws)
    assert '待修复结论' in report
    assert '无待办, 可交付' not in report

def test_report_endpoint_is_readonly_protected_and_uncached(project, monkeypatch):
    store, ws = project
    client = TestClient(app_module.create_app(store.root))
    endpoint = f'/api/projects/{ws["id"]}/report'
    state = (store.folder(ws['id']) / 'state.json').read_bytes()
    response = client.get(endpoint, headers={'Origin': 'https://example.invalid'})
    assert response.status_code == 200
    assert response.headers['cache-control'] == 'no-store'
    assert response.json() == {'revision': ws['revision'], 'report': build_report(ws)}
    assert (store.folder(ws['id']) / 'state.json').read_bytes() == state
    monkeypatch.setenv('ZHILIAN_ACCESS_PASSWORD', 'report-password')
    assert client.get(endpoint).status_code == 401
    assert client.get(endpoint, auth=('zhilian', 'wrong')).status_code == 401
    assert client.get(endpoint, auth=('zhilian', 'report-password')).status_code == 200
    assert client.post(endpoint, auth=('zhilian', 'report-password')).status_code == 405

def test_archive_report_matches_without_changing_file_set(project):
    store, ws = project
    ws = confirm_all(store, ws)
    ws = store.change(ws['id'], ws['revision'], {'sales_current': 90})
    ws = store.repair(ws['id'], ws['revision'], [r['claim_id'] for r in ws['checks'] if r['status'] == 'inconsistent'
    ])

```

```

    expected = build_report(ws)
    with ZipFile(store.archive(ws['id'])) as archive:
        assert set(archive.namelist()) == {d['name'] for d in ws['documents']} | {'核验收报告.md', '知链核验收记录.json'}

    assert archive.read('核验收报告.md').decode('utf-8') == expected
    assert build_report(store.read(ws['id'])) == expected
def test_imported_data_history_is_included(project, tmp_path):
    store, ws = project
    raw = store.read(ws['id'])
    source = next(d for d in raw['documents'] if d['kind'] == 'xlsx')
    wb = load_workbook(store.folder(ws['id']) / raw['generation'] / source['stored_name'])
    wb.active['E3'] = 90
    incoming = tmp_path / 'updated.xlsx'
    wb.save(incoming)
    wb.close()
    ws = store.import_source(ws['id'], ws['revision'], incoming)
    report = build_report(ws)
    assert 'sales_current: 125.0 → 90.0' in report and '事实表!E3' in report
@pytest.mark.parametrize('case,title', [('missing', '数据缺失'), ('count', '来源冲突'),
                                          ('scope', '口径不一致'), ('zero', '需人工复核')])
def test_unverifiable_todos_are_grouped(project, case, title):
    store, ws = project
    raw = store.read(ws['id'])
    claim = next(c for c in raw['claims'] if c['kind'] == ('quote' if case in ('missing', 'count') else 'growth'))
    if case == 'missing':
        claim['refs'] = []

    elif case == 'count':
        claim['refs'] = ['sales_prev', 'sales_current']
    elif case == 'scope':
        claim['refs'].reverse()
    else:
        next(f for f in raw['facts'] if f['id'] == 'sales_prev')['value'] = 0
    report = build_report(raw)
    assert f'### 无法判断 • {title}' in report
    assert f'- [ ] 无法判断: {claim["original"]}' in report
def test_ocr_todos_are_present_without_double_counting(project):
    store, ws = project
    ws = confirm_all(store, ws)
    raw = store.read(ws['id'])
    image_claim = dict(next(c for c in raw['claims'] if c['kind'] == 'quote'),
                       id='image-claim', source='ocr', image_id='image', confirmed=False, repairable=False)
    raw['ocr_claims'] = [image_claim]
    before = deepcopy(raw)
    report = build_report(raw)
    assert '检查论断 12 项' in report and '来源待确认 1 项' in report
    assert report.count('- [ ] 图片论断（只读）') == 1
    assert '需编辑原图后重新导入' in report
    assert '无待办，可交付' not in report
    assert report == build_report(store.public(raw))
    assert raw == before
def test_external_model_text_is_not_in_report(project):
    _, ws = project
    ws['diagnosis_explanations'] = {'anything': 'test-secret provider-error'}
    ws['audit'].append({'time': 'now', 'event': '模型错误', 'detail': 'test-secret provider-error'})

```

```
ws['agent'] = {'error': 'test-secret provider-error'}
report = build_report(ws)
assert 'test-secret' not in report and 'provider-error' not in report
// ===== 文件: tests/test_reranker.py =====
from zhilian import reranker
def test_local_reranker_is_opt_in(monkeypatch):
    monkeypatch.delenv('ZHILIAN_LOCAL_RERANKER_PATH', raising=False)
    monkeypatch.delenv('ZHILIAN_LOCAL_RERANKER_ENABLED', raising=False)
    assert reranker.status()['enabled'] is False
def test_reranker_abstains_when_margin_is_small():
    scores = [
        {'fact_id': 'a', 'score': 0.2, 'raw_score': -1.0},
        {'fact_id': 'b', 'score': 0.19, 'raw_score': -1.2},
    ]
    result = reranker.choose(scores, min_score=-2.0, min_margin=0.3)
    assert result['action'] == 'abstain'
def test_reranker_accepts_calibrated_candidate():
    scores = [
        {'fact_id': 'a', 'score': 0.2, 'raw_score': -1.0},
        {'fact_id': 'b', 'score': 0.1, 'raw_score': -1.4},
    ]
    result = reranker.choose(scores, min_score=-2.0, min_margin=0.3)

    assert result['action'] == 'link'
    assert result['refs'] == ['a']
// ===== 文件: tests/test_review_agents.py =====
from zhilian.demo import create_demo
from zhilian.store import Store
from zhilian import agent
from zhilian import review
def approve_all(store, ws):
    return store.confirm(ws['id'], ws['revision'], [
        {'claim_id': c['id'], 'refs': c['refs']} for c in ws['claims']
    ])
def test_cross_document_audit_is_read_only_evidence(tmp_path):
    store = Store(tmp_path / 'data')
    ws = store.create('跨文档审计', create_demo(tmp_path / 'files'))
    ws = approve_all(store, ws)
    revision = ws['revision']
    result = store.cross_document_audit(ws['id'], revision)
    assert result['cross_audit']['signature']
    assert result['cross_audit']['groups']
    assert result['revision'] == revision
    assert result['cross_audit']['findings'] == []
    assert not result.get('last_repair')
def test_repair_plan_contains_risk_and_does_not_write(tmp_path):
    store = Store(tmp_path / 'data')
    ws = store.create('修复规划', create_demo(tmp_path / 'files'))
    ws = approve_all(store, ws)
    ws = store.change(ws['id'], ws['revision'], {'sales_current': 90})
    result = store.build_repair_plan(ws['id'], ws['revision'])
    assert result['repair_plan']['requires_approval'] is True
    assert result['repair_plan']['summary']['total'] > 0
    item = result['repair_plan']['items'][0]
    assert {'file_id', 'location', 'before', 'after', 'risk_level', 'verification'} <= item.keys()
```

```

    assert result['last_repair'] is None
def test_review_plan_becomes_stale_after_data_change(tmp_path):
    store = Store(tmp_path / 'data')
    ws = store.create('审计快照', create_demo(tmp_path / 'files'))
    raw = store.read(ws['id'])
    raw['cross_audit'] = review.audit_documents(raw)
    store.write(raw)
    ws = store.change(raw['id'], raw['revision'], {'sales_current': 90})
    assert ws['cross_audit']['stale'] is True
    assert ws['cross_audit']['signature'] != review.signature(ws)
def test_semantic_audit_is_metadata_only_and_logged(tmp_path, monkeypatch):
    monkeypatch.setenv('DEEPSEEK_API_KEY', 'test-secret')
    store = Store(tmp_path / 'data')
    ws = store.create('语义审计', create_demo(tmp_path / 'files'))
    ws = approve_all(store, ws)
    seen = {}
    def fake_semantic(groups):
        seen['groups'] = groups

        claims = groups[0]['claims']
        by_file = {}
        for item in claims:
            by_file.setdefault(item['file_id'], item['claim_id'])
        return [{'claim_ids': list(by_file.values())[:2],
                'category': 'semantic_scope_risk', 'reason': '不同文件对同一业务指标使用了不同表述'}]
    monkeypatch.setattr('zhilian.llm.suggest_cross_document_findings', fake_semantic)
    result = store.cross_document_audit(ws['id'], ws['revision'])
    audit = result['cross_audit']
    assert audit['semantic_status'] == 'completed'
    assert any(item['source'] == 'model' for item in audit['findings'])
    assert result['model_calls'][-1]['node'] == 'audit_agent'
    assert all('value' not in item for group in seen['groups'] for item in group['claims'])
    assert all('test-secret' not in str(item) for item in seen['groups'])
def test_review_agent_endpoints_leave_trace_and_files_unchanged(tmp_path):
    from zhilian import agent
    store = Store(tmp_path / 'data')
    ws = store.create('审计智能体', create_demo(tmp_path / 'files'))
    ws = approve_all(store, ws)
    before = {d['id']: (store.folder(ws['id']) / store.read(ws['id'])['generation'] /
                       next(x['stored_name'] for x in store.read(ws['id'])['documents'] if x['id'] == d['id'])).read_bytes()
              for d in ws['documents']}
    ws = agent.run_cross_document_audit(store, ws['id'], ws['revision'])
    ws = agent.run_repair_plan(store, ws['id'], ws['revision'])
    nodes = [entry['node'] for entry in ws['agent']['trace']]
    assert 'cross_document_audit' in nodes and 'plan_repair' in nodes
    current = store.read(ws['id'])
    for document in current['documents']:
        assert (store.folder(ws['id']) / current['generation'] / document['stored_name']).read_bytes() == before[document['id']]
// ===== 文件: tests/test_rule_boundary.py =====
"""Natural-language boundary checks for the deterministic rule matcher.
The ordinary workflow fixtures intentionally use fact-table vocabulary. This
set changes only the wording (营收/销售收入/费用/出货量等), so a high score
cannot be obtained by copying the metric strings from the spreadsheet.

```

```

"""
from pathlib import Path
from zhilian import engine, office
ROOT = Path(__file__).resolve().parents[1]
FACTS = office.read_facts(ROOT / '长文Word测试' / '配套数据_初始.xlsx', 'facts')
CASES = [
    ('本期销售额为125万元', ['sales_current']),
    ('本期销售额累计125万元', ['sales_current']),
    ('本报告期销售额为125万元', ['sales_current']),
    ('报告期内销售金额为125万元', ['sales_current']),
    ('本期销售收入为125万元', ['sales_current']),
    ('本期营业总收入达125万元', ['sales_current']),
    ('公司本期实现营收125万元', ['sales_current']),
    ('本期销售业绩为125万元', ['sales_current']),
    ('A产品销售量为80件', ['product_a']),
    ('A产品出货量达80件', ['product_a']),

    ('本期开支为80万元', ['spending']),
    ('本期费用为80万元', ['spending']),
    ('本期计划金额为100万元', ['budget']),
    ('本期预算为100万元', ['budget']),
    ('本期销售额超过120万元', ['sales_current']),
    ('报告期销售额突破120万元', ['sales_current']),
    ('A产品销量最高', ['product_a', 'product_b']),
    ('A产品销售量为各产品之最', ['product_a', 'product_b']),
]

def test_rule_boundary_uses_rewritten_language_set():
    picks = [[f['id'] for f in engine.best_facts(text, FACTS)] for text, _ in CASES]
    gold = [expected for _, expected in CASES]
    recall = sum(bool(pick) for pick in picks) / len(CASES)
    top1 = sum(pick == expected for pick, expected in zip(picks, gold)) / len(CASES)
    # These are baseline measurements, not a claim that rules understand
    # synonyms. A future alias table or semantic layer may improve them.
    assert round(recall, 4) == 0.3889
    assert round(top1, 4) == 0.3333

def test_period_synonym_is_deterministically_narrowed():
    assert [f['id'] for f in engine.best_facts('本报告期销售额为125万元', FACTS)] == ['sales_current']
    assert [f['id'] for f in engine.best_facts('报告期销售额突破120万元', FACTS)] == ['sales_current']

// ===== 文件: tests/test_source_preview.py =====
from copy import deepcopy
import pytest
from openpyxl import load_workbook
from zhilian.demo import create_demo
from zhilian.engine import check
from zhilian.store import Store

def confirmed_project(tmp_path):
    store = Store(tmp_path / 'data')
    store.create('预览项目', create_demo(tmp_path / 'files'), True)
    state = store.read(store.list()[0]['id'])
    result = store.confirm(state['id'], state['revision'],
                           [{'claim_id': c['id'], 'refs': c['refs']} for c in state['claims']])
    return store, result

def updated_book(tmp_path, value=123):
    path = create_demo(tmp_path / 'external')[0]
    wb = load_workbook(path)

```

```

        wb.active['E3'] = value
        wb.save(path)
        wb.close()
        return path
def test_preview_is_read_only_and_keeps_true_claims(tmp_path):
    store, ws = confirmed_project(tmp_path)
    before = store.read(ws['id'])
    generation = before['generation']
    files = {p.name: p.stat().st_mtime_ns for p in (store.folder(ws['id']) / generation).iterdir()}
    result = store.preview_source(ws['id'], ws['revision'], updated_book(tmp_path, 123))
    assert result['revision'] == ws['revision']
    assert result['summary']['changed_facts'] == 1

    assert result['summary']['affected_claims'] > 0
    assert any(a['after_status'] == 'inconsistent' for a in result['affected'])
    assert all(a['claim_id'] in {c['id'] for c in ws['claims']} for a in result['affected'])
    assert store.read(ws['id']) == before
    assert {p.name: p.stat().st_mtime_ns for p in (store.folder(ws['id']) / generation).iterdir()} == files
def test_preview_rejects_conflict_and_semantic_change(tmp_path):
    store, ws = confirmed_project(tmp_path)
    path = updated_book(tmp_path, 123)
    with pytest.raises(ValueError, match='其他窗口'):
        store.preview_source(ws['id'], ws['revision'] - 1, path)
    wb = load_workbook(path)
    wb.active['G3'] = '其他口径'
    wb.save(path)
    wb.close()
    with pytest.raises(ValueError, match='统计口径'):
        store.preview_source(ws['id'], ws['revision'], path)
def test_preview_matches_applied_import_checks(tmp_path):
    store, ws = confirmed_project(tmp_path)
    path = updated_book(tmp_path, 123)
    preview = store.preview_source(ws['id'], ws['revision'], path)
    updated = store.import_source(ws['id'], ws['revision'], path)
    actual = {c['id']: check(c, updated['facts']) for c in updated['claims']}
    assert {a['claim_id']: a['after_status'] for a in preview['affected']} == {
        cid: actual[cid]['status'] for cid in (a['claim_id'] for a in preview['affected'])
    }
// ===== 文件: tests/test_tools.py =====
import json
from openpyxl import load_workbook
from zhilian import tools
from zhilian.demo import create_demo
from zhilian.mcp_server import build_mcp
from zhilian.store import Store
EXPECTED = {'list_projects', 'parse_facts', 'parse_document', 'verify_claim',
            'verify_project', 'diagnose_project', 'diff_excel'}
def test_tool_registry_and_mcp_registration():
    assert len(tools.TOOLS) == 7
    assert {item['name'] for item in tools.TOOLS} == EXPECTED
    server = build_mcp()
    assert {item.name for item in server._tool_manager.list_tools()} == EXPECTED
def test_read_only_tools_and_json_serialization(tmp_path):
    paths = create_demo(tmp_path / 'files')
    store = Store(tmp_path / 'data')

```

```

ws = store.create('工具测试', paths, demo=True)
listed = tools.invoke(store, 'list_projects', {})
assert listed['projects'][0]['id'] == ws['id']
assert {'id', 'name', 'revision'} <= set(listed['projects'][0])
facts = tools.invoke(store, 'parse_facts', {'excel_path': str(paths[0])})
parsed = tools.invoke(store, 'parse_document', {
    'excel_path': str(paths[0]), 'document_path': str(paths[1])})
assert facts['facts']

assert parsed['claims']
assert isinstance(parsed['warnings'], list)
checked = tools.invoke(store, 'verify_claim', {
    'claim': ws['claims'][0], 'facts': ws['facts']})
assert checked['status'] in {'consistent', 'inconsistent', 'unverifiable'}
assert {'code', 'evidence'} <= set(checked)
verified = tools.invoke(store, 'verify_project', {'project_id': ws['id']})
diagnosed = tools.invoke(store, 'diagnose_project', {'project_id': ws['id']})
assert verified['summary']['claims'] > 0
assert isinstance(diagnosed['records'], list)
assert {'total', 'consistent', 'inconsistent'} <= set(diagnosed['summary'])
changed = tmp_path / 'changed.xlsx'
changed.write_bytes(paths[0].read_bytes())
workbook = load_workbook(changed)
workbook.active['E3'] = 90
workbook.save(changed)
diff = tools.invoke(store, 'diff_excel', {'project_id': ws['id'], 'excel_path': str(changed)})
assert diff['changes']
assert diff['summary']['affected_claims'] >= 0
assert store.read(ws['id'])['revision'] == ws['revision']
for name in EXPECTED:
    params = {
        'list_projects': {},
        'parse_facts': {'excel_path': str(paths[0])},
        'parse_document': {'excel_path': str(paths[0]), 'document_path': str(paths[1])},
        'verify_claim': {'claim': ws['claims'][0], 'facts': ws['facts']},
        'verify_project': {'project_id': ws['id']},
        'diagnose_project': {'project_id': ws['id']},
        'diff_excel': {'project_id': ws['id'], 'excel_path': str(changed)},
    }[name]
    json.dumps(tools.invoke(store, name, params), ensure_ascii=False)
def test_handlers_return_structured_errors(tmp_path):
    store = Store(tmp_path / 'data')
    assert 'error' in tools.invoke(store, 'parse_facts', {'excel_path': str(tmp_path / 'missing.xlsx')})
    assert 'error' in tools.invoke(store, 'verify_project', {'project_id': '0' * 32})
// ===== 文件: tests/test_workbook_fidelity.py =====
"""事实表更新必须保留图表等未建模部件（openpyxl 的 load+save 会静默丢弃）。
背景: office.update_workbook 原先用 openpyxl load_workbook → 改单元格 → save。
openpyxl 只建模它支持的部分，**图表、条件格式、数据透视表、图片在保存时全部丢失**，
而导出接口返回成功、没有任何警告。中小企业的事实表常自带柱状图/饼图，用户一打开
发现图没了就会判定软件不可用。
现改为 xlsx (zip) 级定点改写：只替换目标单元格的 <v>，其余条目原样拷贝。
本测试用真实带图表的 xlsx 验证“图表仍在”与“数值确实被改”。
"""

from __future__ import annotations
import zipfile

```

```

import pytest
from openpyxl import load_workbook
from openpyxl.chart import BarChart, PieChart, Reference
from openpyxl.styles import PatternFill

from zhilian.office import update_workbook
def make_book(path, with_charts=True, with_fill=True):
    wb = load_workbook() if False else None # 显式建新簿，避免依赖模板
    from openpyxl import Workbook
    wb = Workbook()
    ws = wb.active
    ws.title = '事实表'
    ws.append(['事实ID', '主体', '指标', '期间', '数值', '单位', '统计口径'])
    ws.append(['sales_current', '总计', '销售额', '本期', 125, '万元', '演示业务'])
    ws.append(['budget', '总计', '预算', '本期', 100, '万元', '演示项目'])
    if with_fill:
        ws['E2'].fill = PatternFill('solid', fgColor='FFEEAA')
    if with_charts:
        bar = BarChart()
        bar.title = '销售额'
        bar.add_data(Reference(ws, min_col=5, min_row=1, max_row=3), titles_from_data=True)
        ws.add_chart(bar, 'H2')
        pie = PieChart()
        pie.title = '预算占比'
        pie.add_data(Reference(ws, min_col=5, min_row=1, max_row=3), titles_from_data=True)
        ws.add_chart(pie, 'H18')
    wb.save(path)
    return path
def chart_parts(path):
    with zipfile.ZipFile(path) as z:
        return sorted(n for n in z.namelist() if n.startswith('xl/charts/') and n.endswith('.xml'))
def sheet_xml(path, name='xl/worksheets/sheet1.xml'):
    with zipfile.ZipFile(path) as z:
        return z.read(name).decode('utf-8')
def test_update_workbook_keeps_charts_and_styles(tmp_path):
    src = make_book(tmp_path / 'src.xlsx')
    before_charts = chart_parts(src)
    assert len(before_charts) == 2, '测试夹具应含柱状图与饼图各一个'
    dst = tmp_path / 'out.xlsx'
    facts = [{'id': 'sales_current', 'sheet': '事实表', 'cell': 'E2'},
             {'id': 'budget', 'sheet': '事实表', 'cell': 'E3'}]
    update_workbook(src, dst, facts, {'sales_current': 90, 'budget': 110})
    # 1) 图表必须还在——这正是 openpyxl load+save 会丢掉的东西
    after_charts = chart_parts(dst)
    assert after_charts == before_charts, 'Excel 图表在更新后丢失'
    # 2) 数值必须真的被改
    wb = load_workbook(dst)
    ws = wb['事实表']
    assert float(ws['E2'].value) == 90.0
    assert float(ws['E3'].value) == 110.0
    wb.close()
    # 3) 单元格样式也应当保留：填充色记录在 xl/styles.xml (sheet XML 里只存样式索引)
    with zipfile.ZipFile(dst) as z:
        styles = z.read('xl/styles.xml').decode('utf-8')
    assert 'FFEEAA' in styles, '单元格样式在更新后丢失'

```

```

    assert 's=' in sheet_xml(dst), '单元格的样式索引被抹掉了'
def test_update_workbook_does_not_touch_unrelated_cells(tmp_path):
    src = make_book(tmp_path / 'src.xlsx', with_charts=False, with_fill=False)
    dst = tmp_path / 'out.xlsx'
    facts = [{'id': 'sales_current', 'sheet': '事实表', 'cell': 'E2'}]
    update_workbook(src, dst, facts, {'sales_current': 77})
    wb = load_workbook(dst)
    ws = wb['事实表']
    assert float(ws['E2'].value) == 77.0
    assert float(ws['E3'].value) == 100.0    # 未列入 changes, 必须原样
    assert ws['A2'].value == 'sales_current'
    wb.close()
def test_update_workbook_without_changes_copies_file(tmp_path):
    src = make_book(tmp_path / 'src.xlsx')
    dst = tmp_path / 'out.xlsx'
    update_workbook(src, dst, [{'id': 'sales_current', 'sheet': '事实表', 'cell': 'E2'}], {})
    assert chart_parts(dst) == chart_parts(src)
def test_update_workbook_rejects_unresolvable_target(tmp_path):
    """宁可报错, 也不要静默产出残缺文件。"""
    src = make_book(tmp_path / 'src.xlsx')
    dst = tmp_path / 'out.xlsx'
    with pytest.raises(ValueError):
        update_workbook(src, dst, [{'id': 'x', 'sheet': '不存在的表', 'cell': 'E2'}], {'x': 1})
    with pytest.raises(ValueError):
        update_workbook(src, dst, [{'id': 'x', 'sheet': '事实表', 'cell': 'ZZ99'}], {'x': 1})
    assert not dst.exists(), '失败时不应留下半成品文件'
// ===== 文件: tests/test_workflow.py =====
from pathlib import Path
from copy import deepcopy
from zipfile import ZipFile
import pytest
from docx import Document
from fastapi.testclient import TestClient
from openpyxl import load_workbook
from pptx import Presentation
from zhilian.demo import create_demo
from zhilian.engine import check, extract_claims
from zhilian.office import read_facts
from zhilian.store import Store
@pytest.fixture
def project(tmp_path):
    store = Store(tmp_path / 'data')
    w = store.create('测试项目', create_demo(tmp_path / 'files'), True)
    return store, w
def confirm_all(store, w):
    return store.confirm(w['id'], w['revision'], [{'claim_id': c['id'], 'refs': c['refs']} for c in w['claims']])
def test_delete_project_removes_workspace_and_all_versions(project, tmp_path):
    store, w = project
    folder = store.folder(w['id'])
    assert folder.is_dir()

    result = store.delete(w['id'])
    assert result == {'id': w['id'], 'name': '测试项目', 'deleted': True}

```

```

    assert not folder.exists()
    assert store.list() == []
    with pytest.raises(FileNotFoundError):
        store.read(w['id'])
def test_real_files_roundtrip_and_undo(project):
    store, w = project
    assert w['summary']['claims'] >= 11
    assert w['summary']['inconsistent'] == 0
    w = confirm_all(store, w)
    w = store.change(w['id'], w['revision'], {'sales_current': 90, 'product_a': 60, 'spending': 110})
    assert w['summary']['inconsistent'] == 11
    ids = [c['id'] for c in w['claims'] if check(c, w['facts'])['status'] == 'inconsistent']
    w = store.repair(w['id'], w['revision'], ids)
    assert w['summary']['inconsistent'] == 0
    assert w['last_repair']['verified']
    state = store.read(w['id'])
    root = store.folder(w['id']) / state['generation']
    word = next(d for d in state['documents'] if d['kind'] == 'docx')
    doc = Document(root / word['stored_name'])
    text = '\n'.join(p.text for p in doc.paragraphs)
    assert '本期销售额为90万元' in text
    assert '较上期下降10%' in text
    assert 'B产品销量最高' in text
    assert '支出超过预算' in text
    assert '本段是人工撰写的背景说明，修复时应保持原样。' in text
    para = next(p for p in doc.paragraphs if '本期销售额为90' in p.text)
    assert para.runs[0].bold is True
    # Same-text replacements retain all unaffected runs; inserted text adopts its first run.
    deck = next(d for d in state['documents'] if d['kind'] == 'pptx')
    prs = Presentation(root / deck['stored_name'])
    chart = next(s.chart for slide in prs.slides for s in slide.shapes if s.has_chart)
    assert list(chart.series[0].values) == [100, 90]
    archive = store.archive(w['id'])
    with ZipFile(archive) as z:
        assert {'业务数据.xlsx', '分析报告.docx', '业务汇报.pptx', '核验报告.md', '知链核验记录.json'} <= set(z.name
list())
    w = store.undo(w['id'], w['revision'])
    assert w['summary']['inconsistent'] == 11
    w = store.undo(w['id'], w['revision'])
    assert w['summary']['inconsistent'] == 0
    assert next(f for f in w['facts'] if f['id'] == 'sales_current')['value'] == 125
def test_changed_dependency_can_remain_true(project):
    store, w = project
    w = confirm_all(store, w)
    w = store.change(w['id'], w['revision'], {'sales_current': 123})
    thresholds = [check(c, w['facts']) for c in w['claims'] if c['kind'] == 'threshold']
    assert all(c['status'] == 'consistent' for c in thresholds)
    assert any(check(c, w['facts'])['status'] == 'inconsistent' for c in w['claims'] if c['kind'] == 'growth')
def test_unconfirmed_and_stale_revision_rejected(project):

    store, w = project
    w = store.change(w['id'], w['revision'], {'sales_current': 90})
    with pytest.raises(ValueError, match='来源已确认'):
        store.repair(w['id'], w['revision'], [w['claims'][0]['id']])
    with pytest.raises(ValueError, match='其他窗口'):

```

```

        store.change(w['id'], 0, {'sales_current': 80})
def test_zero_base_and_scope_conflict(project):
    _, w = project
    growth = next(c for c in w['claims'] if c['kind'] == 'growth')
    facts = deepcopy(w['facts'])
    next(f for f in facts if f['id'] == 'sales_prev')['value'] = 0
    assert check(growth, facts)['status'] == 'unverifiable'
    facts = deepcopy(w['facts'])
    next(f for f in facts if f['id'] == 'sales_current')['scope'] = '不同口径'
    assert check(growth, facts)['status'] == 'unverifiable'
def test_ambiguous_source_and_missing_evidence(project):
    _, w = project
    facts = deepcopy(w['facts'])
    duplicate = deepcopy(next(f for f in facts if f['id'] == 'sales_current'))
    duplicate.update(id='other-sales', scope='其他地区')
    claims = extract_claims({'file_id': 'x', 'location': 'l', 'label': 'p', 'text': '本期销售额为125万元。'}, facts
+ [duplicate])
    assert claims[0]['refs'] == []
    assert check(claims[0], facts)['status'] == 'unverifiable'
    assert extract_claims({'file_id': 'x', 'location': 'l', 'label': 'p', 'text': '增长主要来自渠道优化。'}, facts)
== []
def test_tied_ranking(project):
    _, w = project
    rank = next(c for c in w['claims'] if c['kind'] == 'ranking')
    facts = deepcopy(w['facts'])
    next(f for f in facts if f['id'] == 'product_b')['value'] = 80
    r = check(rank, facts)
    assert r['status'] == 'inconsistent'
    assert r['expected'] == 'A产品、B产品销量并列最高'
def test_formula_rejected(tmp_path):
    path = create_demo(tmp_path)[0]
    wb = load_workbook(path)
    wb.active['E2'] = '=SUM(10,90)'
    wb.save(path)
    wb.close()
    with pytest.raises(ValueError, match='公式'):
        read_facts(path, 'f')
def test_disk_tamper_rejected(project):
    store, w = project
    state = store.read(w['id'])
    path = store.folder(w['id']) / state['generation'] / state['documents'][0]['stored_name']
    with path.open('ab') as f:
        f.write(b'changed')
    with pytest.raises(ValueError, match='源文件'):
        store.change(w['id'], w['revision'], {'sales_current': 90})
def test_api_workflow_and_boundary(tmp_path, monkeypatch):
    monkeypatch.delenv('ZHILIAN_ACCESS_PASSWORD', raising=False)

    from zhilian.app import create_app
    client = TestClient(create_app(tmp_path / 'api'))
    assert client.get('/').status_code == 200
    assert client.get('/api/health').json()['status'] == 'ok'
    w = client.post('/api/projects/demo').json()
    assert w['summary']['consistent'] >= 11
    response = client.post(f'/api/projects/{w["id"]}/facts', json={'revision':0,'values':{'sales_current':90}}, head

```

```

ers={'Origin':'https://evil.example'})
    assert response.status_code == 403
    assert client.get('/api/projects/not-valid').status_code == 400
    assert client.get('/api/projects/' + 'a'*32).status_code == 404
    assert client.get(f'/api/projects/{w["id"]}/export').status_code == 200
    deleted = client.delete(f'/api/projects/{w["id"]}')
    assert deleted.status_code == 200 and deleted.json()['deleted'] is True
    assert client.get(f'/api/projects/{w["id"]}').status_code == 404
    monkeypatch.setenv('ZHILIAN_ACCESS_PASSWORD', 'local-test-password')
    assert client.get('/api/health').status_code == 401
    assert client.get('/api/health', auth=('zhilian', 'local-test-password')).status_code == 200
def test_upload_and_persistence(tmp_path, monkeypatch):
    monkeypatch.delenv('ZHILIAN_ACCESS_PASSWORD', raising=False)
    from zhilian.app import create_app
    paths = create_demo(tmp_path / 'input')
    client = TestClient(create_app(tmp_path / 'data'))
    response = client.post('/api/projects', data={'name': '上传项目'}, files=[('files', (p.name, p.read_bytes(), 'application/octet-stream')) for p in paths[:2]])
    assert response.status_code == 200, response.text
    w = response.json()
    assert w['name'] == '上传项目'
    assert w['summary']['documents'] == 2
    second_client = TestClient(create_app(tmp_path / 'data'))
    assert second_client.get(f'/api/projects/{w["id"]}').json()['summary']['claims'] == 5
def test_append_document_batch_keeps_one_excel_and_scans_new_files(tmp_path):
    store = Store(tmp_path / 'data')
    initial = create_demo(tmp_path / 'initial')
    w = store.create('批量追加测试', initial[:2])
    extra = create_demo(tmp_path / 'extra')[1]
    extra_copy = tmp_path / 'extra' / '第二份报告.docx'
    doc = Document(extra)
    doc.add_paragraph('追加批次文档中的新内容')
    doc.save(extra_copy)
    before = store.read(w['id'])
    updated = store.append_documents(w['id'], before['revision'], [extra_copy])
    state = store.read(w['id'])
    assert len([d for d in state['documents'] if d['kind'] == 'xlsx']) == 1
    assert len(state['documents']) == 3
    assert updated['batch']['status'] == 'done'
    new_id = next(d['id'] for d in state['documents'] if d['name'] == extra_copy.name)
    assert any(c['file_id'] == new_id for c in state['claims'])
    assert store.batch(w['id'], updated['batch']['id'])['count'] == 1
    with pytest.raises(ValueError, match='同名文件'):
        store.append_documents(w['id'], updated['revision'], [extra_copy])
def test_append_documents_api_rejects_excel_and_supports_batch(tmp_path, monkeypatch):
    monkeypatch.delenv('ZHILIAN_ACCESS_PASSWORD', raising=False)
    from zhilian.app import create_app
    app = create_app(tmp_path / 'api')
    client = TestClient(app)
    paths = create_demo(tmp_path / 'input')
    response = client.post('/api/projects', data={'name': '批次 API'},
                           files=[('files', (p.name, p.read_bytes(), 'application/octet-stream')) for p in paths[:2]
    ])
    assert response.status_code == 200, response.text

```

```

w = response.json()
extra = tmp_path / 'input' / '追加报告.docx'
doc = Document(paths[1])
doc.add_paragraph('API 追加批次的新内容')
doc.save(extra)
response = client.post(f'/api/projects/{w["id"]}/documents', data={'revision': w['revision']},
                        files=[('files', (extra.name, extra.read_bytes(), 'application/octet-stream'))])
assert response.status_code == 200, response.text
result = response.json()
assert result['batch']['count'] == 1
assert result['summary']['documents'] == 3
batch = client.get(f'/api/projects/{w["id"]}/batches/{result["batch"]["id"]}')
assert batch.status_code == 200 and batch.json()['status'] == 'done'
excel = paths[0]
rejected = client.post(f'/api/projects/{w["id"]}/documents', data={'revision': result['revision']},
                        files=[('files', (excel.name, excel.read_bytes(), 'application/octet-stream'))])
assert rejected.status_code == 400
def test_append_documents_api_tolerant_mode_keeps_good_files_and_reports_bad(tmp_path, monkeypatch):
    monkeypatch.delenv('ZHILIAN_ACCESS_PASSWORD', raising=False)
    from zhilian.app import create_app
    app = create_app(tmp_path / 'api')
    client = TestClient(app)
    paths = create_demo(tmp_path / 'input')
    created = client.post('/api/projects', data={'name': '容错批次'},
                          files=[('files', (p.name, p.read_bytes(), 'application/octet-stream')) for p in paths[:2]])
    )
    assert created.status_code == 200, created.text
    w = created.json()
    good = tmp_path / 'input' / '追加正常.docx'
    doc = Document(paths[1])
    doc.add_paragraph('容错模式应保留的正常文件')
    doc.save(good)
    bad = tmp_path / 'input' / '追加损坏.docx'
    bad.write_bytes(b'not an Office document')
    response = client.post(
        f"/api/projects/{w['id']}/documents",
        data={'revision': w['revision'], 'continue_on_error': 'true'},
        files=[
            ('files', (good.name, good.read_bytes(), 'application/vnd.openxmlformats-officedocument.wordprocessingml.document')),
            ('files', (bad.name, bad.read_bytes(), 'application/vnd.openxmlformats-officedocument.wordprocessingml.document')),
        ],
    )
    assert response.status_code == 200, response.text

    result = response.json()
    assert result['batch']['status'] == 'partial'
    assert result['batch']['count'] == 1 and result['batch']['rejected'] == 1
    assert result['batch_results']['accepted'][0]['name'] == good.name
    assert result['batch_results']['rejected'][0]['name'] == bad.name
    assert '有效的 Office' in result['batch_results']['rejected'][0]['reason']
    assert result['summary']['documents'] == 3
    assert result['revision'] == w['revision'] + 1
def test_append_documents_api_tolerant_mode_rejects_duplicate_but_accepts_new_file(tmp_path, monkeypatch):

```

```

monkeypatch.delenv('ZHILIAN_ACCESS_PASSWORD', raising=False)
from zhilian.app import create_app
app = create_app(tmp_path / 'api')
client = TestClient(app)
paths = create_demo(tmp_path / 'input')
created = client.post('/api/projects', data={'name': '重复文件隔离'},
                      files=[('files', (p.name, p.read_bytes(), 'application/octet-stream')) for p in paths[:2]]
)
w = created.json()
duplicate = paths[1]
good = tmp_path / 'input' / '第三份报告.docx'
doc = Document(paths[1])
doc.add_paragraph('不应被重复文件失败拖累')
doc.save(good)
response = client.post(
    f"/api/projects/{w['id']}/documents",
    data={'revision': w['revision'], 'continue_on_error': 'true'},
    files=[
        ('files', (duplicate.name, duplicate.read_bytes(), 'application/vnd.openxmlformats-officedocument.wordpr
ocessingml.document')),
        ('files', (good.name, good.read_bytes(), 'application/vnd.openxmlformats-officedocument.wordprocessingml
.document')),
    ],
)
assert response.status_code == 200, response.text
result = response.json()
assert result['batch']['status'] == 'partial'
assert result['batch']['count'] == 1 and result['batch']['rejected'] == 1
assert result['batch_results']['rejected'][0]['name'] == duplicate.name
assert result['batch_results']['accepted'][0]['name'] == good.name
assert result['summary']['documents'] == 3
def test_append_documents_api_tolerant_mode_all_bad_does_not_change_project(tmp_path, monkeypatch):
    monkeypatch.delenv('ZHILIAN_ACCESS_PASSWORD', raising=False)
    from zhilian.app import create_app
    app = create_app(tmp_path / 'api')
    client = TestClient(app)
    paths = create_demo(tmp_path / 'input')
    created = client.post('/api/projects', data={'name': '全失败批次'},
                          files=[('files', (p.name, p.read_bytes(), 'application/octet-stream')) for p in paths[:2]]
    )
    w = created.json()
    bad = tmp_path / 'input' / '全损坏.docx'
    bad.write_bytes(b'not an Office document')
    response = client.post(
        f"/api/projects/{w['id']}/documents",

        data={'revision': w['revision'], 'continue_on_error': 'true'},
        files=[('files', (bad.name, bad.read_bytes(), 'application/octet-stream'))],
    )
    assert response.status_code == 200, response.text
    result = response.json()
    assert result['batch']['status'] == 'failed'
    assert result['batch']['count'] == 0 and result['batch']['rejected'] == 1
    assert result['revision'] == w['revision']
    assert result['summary']['documents'] == w['summary']['documents']

```

```

def test_append_document_batch_rolls_back_when_one_file_is_invalid(tmp_path):
    store = Store(tmp_path / 'data')
    paths = create_demo(tmp_path / 'files')
    w = store.create('追加回滚测试', paths[:2])
    invalid = tmp_path / 'files' / '损坏.docx'
    invalid.write_bytes(b'not an Office document')
    before = store.read(w['id'])
    with pytest.raises(ValueError, match='有效的 Office'):
        store.append_documents(w['id'], before['revision'], [invalid])
    after = store.read(w['id'])
    assert after['revision'] == before['revision']
    assert [d['name'] for d in after['documents']] == [d['name'] for d in before['documents']]

def test_external_excel_import_repair_and_undo(tmp_path, monkeypatch):
    monkeypatch.delenv('ZHILIAN_ACCESS_PASSWORD', raising=False)
    from zhilian.app import create_app
    app = create_app(tmp_path / 'data')
    client = TestClient(app)
    w = client.post('/api/projects/demo').json()
    w = confirm_all(app.state.store, w)
    path = create_demo(tmp_path / 'external')[0]
    wb = load_workbook(path)
    wb.active['E3'] = 90
    wb.save(path)
    wb.close()
    endpoint = f'/api/projects/{w["id"]}/source'
    response = client.post(endpoint, data={'revision': w['revision']}, files={'file': (path.name, path.read_bytes())})
    assert response.status_code == 200, response.text
    updated = response.json()
    assert updated['summary']['inconsistent'] > 0
    assert all(c['confirmed'] for c in updated['claims'])
    assert next(f for f in updated['facts'] if f['id'] == 'sales_current')['value'] == 90
    assert any('125' in b['text'] for b in updated['blocks'])
    assert client.post(endpoint, data={'revision': w['revision']}, files={'file': (path.name, path.read_bytes())}).status_code == 400
    ids = [c['id'] for c in updated['claims'] if check(c, updated['facts'])['status'] == 'inconsistent']
    repaired = app.state.store.repair(w['id'], updated['revision'], ids)
    assert repaired['summary']['inconsistent'] == 0
    assert any('下降10%' in b['text'] for b in repaired['blocks'])
    restored = app.state.store.undo(w['id'], repaired['revision'])
    restored = app.state.store.undo(w['id'], restored['revision'])
    assert next(f for f in restored['facts'] if f['id'] == 'sales_current')['value'] == 125

def test_external_excel_semantic_change_rejected(project, tmp_path):
    store, w = project
    path = create_demo(tmp_path / 'external')[0]
    wb = load_workbook(path)
    wb.active['G3'] = '其他口径'
    wb.active['E3'] = 90
    wb.save(path)
    wb.close()
    with pytest.raises(ValueError, match='统计口径'):
        store.import_source(w['id'], w['revision'], path)
    assert store.read(w['id'])['revision'] == w['revision']

```